EN4554_A02_CNN_220067G

October 20, 2026

EN4554 — Assignment 2: Convolutional Neural Networks

Name	*Bandara RMTD*
Index number	*220067G*
GitHub repository	*https://github.com/dilshan-49/EN4554_A02*

Acknowledgement of AI tool use: I used Claude (Anthropic) to help write the code and to draft the structure of this notebook. I have read, run and understood every cell, tuned the settings myself, and checked every interpretation against the results that I obtained.

Contents

1. Setup: data split, normalisation and shared training utilities

2. Backpropagation
3. Optimizers

4. Regularisation and residual connections

5. Best CNN with at most 500k parameters and 20 epochs, with ablations and a single test-set evaluation

6. Conclusions

Placeholders. Each interpretation cell explains the mechanism behind the expected result. Numbers written as **[??]** must be filled in from your own run, and any statement that your plots do not support must be corrected.

1 Setup

[11]:
```
# ------------------------------------------------------------------
# Imports and global configuration
# ------------------------------------------------------------------
import os, re, math, time, copy, random, pickle
from contextlib import nullcontext

import numpy as np
import pandas as pd
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import matplotlib.pyplot as plt
from IPython.display import display

SEED = 42
DATA_ROOT = './data'
# Training results are cached here so that re-running a cell (or recovering␣
 ↪from a Colab disconnect) does not
# repeat the training. Delete a file to force that run to be repeated. On Colab␣
 ↪you can mount Google Drive and
# point this at e.g. '/content/drive/MyDrive/en4554_a2_results' so the cache␣
 ↪survives a runtime reset.

try:                                    # on Colab: keep the cache on Google␣
 ↪Drive so it survives a disconnect
    from google.colab import drive
    drive.mount('/content/drive')
    RESULTS_DIR = '/content/drive/MyDrive/en4554_a2/results'
    DATA_ROOT = '/content/drive/MyDrive/en4554_a2/data'
except ImportError:                     # on the laptop
    RESULTS_DIR = 'results'
os.makedirs(RESULTS_DIR, exist_ok=True)

def pick_device():
    if torch.cuda.is_available():
        return torch.device('cuda')
    if getattr(torch.backends, 'mps', None) is not None and torch.backends.mps.
 ↪is_available():
        return torch.device('mps')          # Apple-silicon laptops
    return torch.device('cpu')

device = pick_device()
USE_AMP = device.type == 'cuda'             # float16 mixed precision on NVIDIA␣
 ↪GPUs only (about 2x faster on a T4)
torch.backends.cudnn.benchmark = True       # cuDNN picks the fastest conv␣
 ↪algorithms (tiny run-to-run noise on GPU)

def seed_everything(seed=SEED):
    """Fix every random number generator so that runs are reproducible."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)                 # seeds the CPU and all CUDA devices
```

```
seed_everything()
plt.rcParams.update({'figure.dpi': 110, 'axes.grid': True, 'grid.alpha': 0.3})
gpu_name = f' ({torch.cuda.get_device_name(0)})' if device.type == 'cuda' else␣
↪''
print(f'PyTorch {torch.__version__} | device: {device}{gpu_name} | mixed␣
↪precision: {USE_AMP}')
```

```
PyTorch 2.6.0+cu118 | device: cuda (NVIDIA GeForce GTX 1650) | mixed precision:
True
```

1.1 Data

The 50,000 CIFAR-10 training images are split once, with a fixed seed, into **45,000 training** and **5,000 validation** images. The per-channel mean and standard deviation are computed on the 45k training images only and are then used to normalise the training, validation and test images, so that no information from the held-out data leaks into preprocessing.

[ ]:
```
# ------------------------------------------------------------------
# CIFAR-10: 45k train / 5k validation split, per-channel normalisation with␣
↪TRAINING statistics
# ------------------------------------------------------------------
cifar_train = torchvision.datasets.CIFAR10(DATA_ROOT, train=True, download=True)
CLASSES = cifar_train.classes

X_all = torch.tensor(cifar_train.data).permute(0, 3, 1, 2).float().div_(255)  ␣
↪# (50000, 3, 32, 32) in [0, 1]
Y_all = torch.tensor(cifar_train.targets, dtype=torch.long)

split_gen = torch.Generator().manual_seed(SEED)                # fixed seed for␣
↪same split every time
perm = torch.randperm(len(X_all), generator=split_gen)
train_idx, val_idx = perm[:45_000], perm[45_000:]
X_tr, Y_tr = X_all[train_idx], Y_all[train_idx]
X_va, Y_va = X_all[val_idx], Y_all[val_idx]
del X_all, cifar_train

MEAN = X_tr.mean(dim=(0, 2, 3))                                # statistics of␣
↪the 45k TRAINING images only
STD = X_tr.std(dim=(0, 2, 3))

def normalize(x):
    """Per-channel standardisation with the training-set mean and std (used for␣
↪train, val and test)."""
    return (x - MEAN.view(1, 3, 1, 1)) / STD.view(1, 3, 1, 1)

# normalise in place to save memory, then move everything to the device
```

```
X_train = X_tr.sub_(MEAN.view(1, 3, 1, 1)).div_(STD.view(1, 3, 1, 1)).to(device)
X_val = X_va.sub_(MEAN.view(1, 3, 1, 1)).div_(STD.view(1, 3, 1, 1)).to(device)
Y_train, Y_val = Y_tr.to(device), Y_va.to(device)
del X_tr, X_va, Y_tr, Y_va


print(f'train {tuple(X_train.shape)}   validation {tuple(X_val.shape)}')
print('training mean per channel:', MEAN.numpy().round(4), '  std:', STD.
 ↪numpy().round(4))
print('after normalisation (train): mean', X_train.mean(dim=(0, 2, 3)).cpu().
 ↪numpy().round(3),
      ' std', X_train.std(dim=(0, 2, 3)).cpu().numpy().round(3))
print('validation class counts:', torch.bincount(Y_val, minlength=10).tolist())
```

```
train (45000, 3, 32, 32)   validation (5000, 3, 32, 32)
training mean per channel: [0.4917 0.4823 0.4467]   std: [0.2471 0.2435 0.2616]
after normalisation (train): mean [0. 0. 0.]  std [1. 1. 1.]
validation class counts: [488, 512, 532, 471, 471, 514, 507, 500, 504, 501]
```

[ ]:
```
def show_rows(rows, n=8):
    """rows: list of (title, images, labels). Undoes the normalisation and␣
 ↪plots n images per row."""
    fig, axes = plt.subplots(len(rows), n, figsize=(1.5 * n, 1.75 * len(rows)),␣
 ↪squeeze=False)
    for r, (title, X, Y) in enumerate(rows):
        imgs = (X[:n].detach().float().cpu() * STD.view(1, 3, 1, 1) + MEAN.
 ↪view(1, 3, 1, 1)).clamp(0, 1)
        for c in range(n):
            ax = axes[r, c]
            ax.imshow(imgs[c].permute(1, 2, 0).numpy())
            ax.set_xticks([]); ax.set_yticks([]); ax.grid(False)
            ax.set_title(CLASSES[int(Y[c])], fontsize=8)
        axes[r, 0].set_ylabel(title, fontsize=8)
    plt.tight_layout();
    plt.show()

show_rows([('training images', X_train, Y_train)], n=10)
```



1.2 Shared utilities

These helpers are used by upcoming tasks:

- `batches` cuts shuffled mini-batches from the on-device tensors.
- `random_flip`, `random_crop`, `cutout` and `mixup` are vectorised GPU implementations of the augmentations for task 3.
- `evaluate` returns the mean cross-entropy and accuracy of a model on a dataset.
- `train` is the generic training loop. It supports augmentation, mixup, label smoothing, a learning-rate schedule and mixed precision, and it records the loss of every iteration as well as the per-epoch training loss, validation loss and accuracy (and optionally the accuracy on clean training images).
- `cached` stores the result of a training run on disk and reloads it the next time.

[14]:
```
# ------------------------------------------------------------------
# Mini-batches and data augmentation
# ------------------------------------------------------------------
def batches(X, Y, batch_size, shuffle=False, drop_last=False):
    """Yield (x, y) mini-batches from tensors that already live on the device.
    """
    N = X.shape[0]
    idx = torch.randperm(N, device=X.device) if shuffle else torch.arange(N,␣
    ↪device=X.device)
    stop = N - N % batch_size if (drop_last and N >= batch_size) else N
    for k in range(0, stop, batch_size):
        j = idx[k:k + batch_size]
        yield X[j], Y[j]

def random_flip(x):
    """Horizontal flip of each image with probability 1/2."""
    flip = torch.rand(x.shape[0], device=x.device) < 0.5
    return torch.where(flip[:, None, None, None], x.flip(3), x)

def random_crop(x, pad=4):
    """Zero-pad by `pad` pixels and take a random 32x32 crop per image (a␣
    ↪random shift of up to +-pad pixels).
    Zero is the mean colour after normalisation."""
    B, C, H, W = x.shape
    xp = F.pad(x, (pad, pad, pad, pad))
    i = torch.randint(0, 2 * pad + 1, (B,), device=x.device)                #␣
    ↪top offset of each crop
    j = torch.randint(0, 2 * pad + 1, (B,), device=x.device)                #␣
    ↪left offset of each crop
    rows = (i[:, None] + torch.arange(H, device=x.device))[:, :, None]      #␣
    ↪(B, H, 1)
    cols = (j[:, None] + torch.arange(W, device=x.device))[:, None, :]      #␣
    ↪(B, 1, W)
    b = torch.arange(B, device=x.device)[:, None, None]                     #␣
    ↪(B, 1, 1)
    return xp.permute(0, 2, 3, 1)[b, rows, cols].permute(0, 3, 1, 2).
    ↪contiguous()
```

```
def cutout(x, size=16):
    """Cutout: zero a size x size square at a random centre in every image.
    The square may extend beyond the border (then a smaller area is erased), as␣
 ↪in the original paper."""
    B, C, H, W = x.shape
    cy = torch.randint(0, H, (B, 1, 1), device=x.device)
    cx = torch.randint(0, W, (B, 1, 1), device=x.device)
    ys = torch.arange(H, device=x.device)[None, :, None]
    xs = torch.arange(W, device=x.device)[None, None, :]
    inside = (ys >= cy - size // 2) & (ys < cy + size - size // 2) & (xs >= cx␣
 ↪- size // 2) & (xs < cx + size - size // 2)
    return x * (~inside)[:, None].to(x.dtype)

def mixup(x, y, alpha):
    """Mixup: convex combination of the batch with a shuffled copy of itself.
    Returns the mixed images, both label vectors and the mixing weight lambda ~␣
 ↪Beta(alpha, alpha)."""
    lam = float(np.random.beta(alpha, alpha))
    perm = torch.randperm(x.shape[0], device=x.device)
    return lam * x + (1 - lam) * x[perm], y, y[perm], lam

def make_augment(flip_crop=False, cutout_size=0):
    """Build the per-batch augmentation function (None if no augmentation)."""
    if not flip_crop and not cutout_size:
        return None
    def augment(x):
        if flip_crop:
            x = random_crop(random_flip(x))
        if cutout_size:
            x = cutout(x, cutout_size)
        return x
    return augment
```

[15]:
```
# ------------------------------------------------------------------
# Evaluation, training loop, learning-rate schedule, caching
# ------------------------------------------------------------------
def autocast_ctx():
    return torch.autocast('cuda', dtype=torch.float16) if USE_AMP else␣
 ↪nullcontext()

def make_grad_scaler():
    return torch.amp.GradScaler('cuda', enabled=USE_AMP)

def count_params(model):
    return sum(p.numel() for p in model.parameters())
```

```
@torch.no_grad()
def evaluate(model, X, Y, batch_size=1000):
    """Mean cross-entropy and accuracy of `model` on (X, Y), in eval mode (BN␣
 ↪uses running stats, no dropout)."""
    model.eval()
    total_loss, correct = 0.0, 0
    for xb, yb in batches(X, Y, batch_size):
        with autocast_ctx():
            out = model(xb)
        total_loss += F.cross_entropy(out.float(), yb, reduction='sum').item()
        correct += (out.argmax(1) == yb).sum().item()
    return total_loss / len(X), correct / len(X)

def cosine_schedule(warmup_frac=0.0):
    """Learning-rate multiplier: linear warm-up over the first warmup_frac of␣
 ↪the steps, then cosine decay to 0."""
    def schedule(step, total):
        warm = int(warmup_frac * total)
        if step < warm:
            return (step + 1) / warm
        return 0.5 * (1 + math.cos(math.pi * (step - warm) / max(1, total -␣
 ↪warm)))
    return schedule

def train(model, optimizer, n_epochs, X, Y, X_val, Y_val, batch_size=128,␣
 ↪augment=None, mixup_alpha=0.0,
          label_smoothing=0.0, lr_schedule=None, eval_train=None, name='',␣
 ↪print_every=None):
    """Generic mini-batch training loop (Tasks 2-4).
      augment         function applied to every training batch, or None
      mixup_alpha     > 0 enables mixup with lambda ~ Beta(alpha, alpha)
      lr_schedule     function(step, total_steps) -> multiplier of the base␣
 ↪learning rate (None = constant)
      eval_train      (X, Y) of CLEAN training images; if given, the training␣
 ↪accuracy is measured on them
                      in eval mode at the end of every epoch
    Returns a dict with per-epoch histories and the loss of every iteration."""
    scaler = make_grad_scaler()
    steps_per_epoch = max(1, len(X) // batch_size)
    total_steps = n_epochs * steps_per_epoch
    base_lrs = [g['lr'] for g in optimizer.param_groups]
    print_every = print_every or max(1, n_epochs // 10)
    hist = dict(train_loss=[], val_loss=[], val_acc=[], train_acc=[],␣
 ↪iter_loss=[],
                steps_per_epoch=steps_per_epoch, diverged=False)
    step, t_start = 0, time.time()
```

```
    for ep in range(n_epochs):
        model.train()
        ep_losses = []
        for xb, yb in batches(X, Y, batch_size, shuffle=True, drop_last=True):
            if lr_schedule is not None:
                for g, lr0 in zip(optimizer.param_groups, base_lrs):
                    g['lr'] = lr0 * lr_schedule(step, total_steps)
            if augment is not None:
                xb = augment(xb)
            if mixup_alpha > 0:
                xb, y_a, y_b, lam = mixup(xb, yb, mixup_alpha)
            with autocast_ctx():
                out = model(xb)
                if mixup_alpha > 0:    # the loss is mixed with the same weight␣
↪as the images
                    loss = lam * F.cross_entropy(out, y_a,␣
↪label_smoothing=label_smoothing) \
                           + (1 - lam) * F.cross_entropy(out, y_b,␣
↪label_smoothing=label_smoothing)
                else:
                    loss = F.cross_entropy(out, yb,␣
↪label_smoothing=label_smoothing)
            optimizer.zero_grad(set_to_none=True)
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
            ep_losses.append(loss.detach())
            step += 1
        ep_losses = torch.stack(ep_losses).float().cpu()        # one GPU->CPU␣
↪transfer per epoch
        hist['iter_loss'] += ep_losses.tolist()
        hist['train_loss'].append(ep_losses.mean().item())
        vl, va = evaluate(model, X_val, Y_val)
        hist['val_loss'].append(vl)
        hist['val_acc'].append(va)
        if eval_train is not None:
            hist['train_acc'].append(evaluate(model, *eval_train)[1])
        diverged = not bool(torch.isfinite(ep_losses).all())
        if (ep + 1) % print_every == 0 or ep == 0 or ep == n_epochs - 1 or␣
↪diverged:
            msg = (f'{name:>26s} | ep {ep + 1:3d}/{n_epochs} | train loss␣
↪{hist["train_loss"][-1]:.4f} '
                   f'| val loss {vl:.4f} | val acc {100 * va:5.2f}%')
            if eval_train is not None:
                msg += f' | train acc {100 * hist["train_acc"][-1]:5.2f}%'
            print(msg + f' | {time.time() - t_start:6.1f}s')
```

```
        if diverged:
            print(f'{name:>26s} | the loss became NaN/inf: the run diverged,␣
 ↪stopping it')
            hist['diverged'] = True
            break
    hist['time'] = time.time() - t_start
    return hist

def slug(s):
    return re.sub(r'[^A-Za-z0-9.+-]+', '_', s).strip('_')

def cached(key, fn):
    """Return the stored result of run `key` if it exists, otherwise compute␣
 ↪fn(), store it and return it."""
    path = os.path.join(RESULTS_DIR, slug(key) + '.pkl')
    if os.path.exists(path):
        print(f'[cache] loaded {path}  (delete it to re-run)')
        with open(path, 'rb') as f:
            return pickle.load(f)
    out = fn()
    with open(path, 'wb') as f:
        pickle.dump(out, f)
    return out

def plot_smoothed_loss(ax, hist, label, k=50, **kw):
    """Plot the per-iteration training loss, smoothed with a moving average of␣
 ↪k iterations, against epochs."""
    y = np.asarray(hist['iter_loss'], dtype=float)
    k = max(1, min(k, len(y) // 5))
    ys = np.convolve(y, np.ones(k) / k, mode='valid')
    xs = (np.arange(len(ys)) + k) / hist['steps_per_epoch']
    ax.plot(xs, ys, label=label, **kw)
```

2 Task 1 — Backpropagation

2.1 1(a)

For a mini-batch of N row vectors stacked in $X \in \mathbb{R}^{N\times d}$, and with $G = \partial L/\partial(\text{output})$ the gradient arriving from the layer above, the layers and their backward passes are:

- **Linear**, $Y = XW + b$ with $W \in \mathbb{R}^{d\times m}$:
  $\dfrac{\partial L}{\partial W} = X^\top G, \qquad \dfrac{\partial L}{\partial b} = \mathbf{1}^\top G, \qquad \dfrac{\partial L}{\partial X} = G\,W^\top.$
- **ReLU**, $Y = \max(0, X)$:
  $\dfrac{\partial L}{\partial X} = G \odot \mathbb{1}[X > 0].$
- **Sigmoid**, $Y = \sigma(X)$:
  $\dfrac{\partial L}{\partial X} = G \odot \sigma(X)\,(1 - \sigma(X)).$

- **Softmax cross-entropy** on logits Z with labels y:
  $p_{ik} = \dfrac{e^{z_{ik}}}{\sum_j e^{z_{ij}}}$, $L = -\dfrac{1}{N}\sum_i \log p_{i,y_i}$, and $\dfrac{\partial L}{\partial Z} = \dfrac{1}{N}\left(P - \text{onehot}(y)\right)$.

The softmax is computed with the log-sum-exp trick (subtract the row maximum before exponentiating), which leaves the result unchanged but avoids overflow. Each layer caches what its backward pass needs (the input of `Linear`, the mask of `ReLU`, the output of `Sigmoid`, the probabilities of the loss). `MLP.backward` applies the chain rule by calling the layers' `backward` methods in reverse order.

[ ]:
```
class Linear:
    """Fully connected layer  y = x W + b  with a hand-written backward pass."""
    def __init__(self, in_features, out_features, init='kaiming', dtype=torch.
↪float32, device=device):
        if init == 'kaiming':     # He et al. (2015): Var(W) = 2/fan_in keeps␣
↪the signal variance constant through ReLUs
            std = math.sqrt(2.0 / in_features)
        elif init == 'xavier':    # Glorot & Bengio (2010): Var(W) = 2/(fan_in␣
↪+ fan_out), for symmetric activations
            std = math.sqrt(2.0 / (in_features + out_features))
        else:
            raise ValueError(init)
        self.W = torch.randn(in_features, out_features, dtype=dtype,␣
↪device=device) * std
        self.b = torch.zeros(out_features, dtype=dtype, device=device)
        self.dW, self.db = torch.zeros_like(self.W), torch.zeros_like(self.b)

    def forward(self, x):
        self.x = x                          # cached: needed for dL/dW
        return x @ self.W + self.b

    def backward(self, grad_out):           # grad_out = dL/dy, shape (N, out)
        self.dW = self.x.T @ grad_out       # (in, out)
        self.db = grad_out.sum(dim=0)       # (out,)
        return grad_out @ self.W.T          # dL/dx, shape (N, in)


class ReLU:
    def forward(self, x):
        self.mask = x > 0
        return x.clamp(min=0)

    def backward(self, grad_out):
        return grad_out * self.mask


class Sigmoid:
    def forward(self, x):
```

```
        self.s = torch.sigmoid(x)
        return self.s

    def backward(self, grad_out):
        return grad_out * self.s * (1 - self.s)


class SoftmaxCrossEntropy:
    """Mean cross-entropy of softmax(logits) against integer labels."""
    def forward(self, logits, y):
        z = logits - logits.max(dim=1, keepdim=True).values          #␣
↪log-sum-exp trick for numerical stability
        log_p = z - z.exp().sum(dim=1, keepdim=True).log()
        self.p, self.y = log_p.exp(), y                               # cached␣
↪for the backward pass
        return -log_p[torch.arange(len(y), device=y.device), y].mean()

    def backward(self):
        N = self.y.shape[0]
        grad = self.p.clone()
        grad[torch.arange(N, device=self.y.device), self.y] -= 1      # P -␣
↪onehot(y)
        return grad / N


class MLP:
    """Multi-layer perceptron built from the layers above: Linear -> act -> ...␣
↪-> Linear -> softmax CE."""
    def __init__(self, sizes, activation='relu', init='kaiming', dtype=torch.
↪float32, device=device):
        self.layers = []
        for i, (d_in, d_out) in enumerate(zip(sizes[:-1], sizes[1:])):
            self.layers.append(Linear(d_in, d_out, init, dtype, device))
            if i < len(sizes) - 2:                                    # no␣
↪activation after the last layer
                self.layers.append(ReLU() if activation == 'relu' else␣
↪Sigmoid())
        self.loss_fn = SoftmaxCrossEntropy()

    def forward(self, x):
        for layer in self.layers:
            x = layer.forward(x)
        return x                                                      # logits

    def loss(self, x, y):
        return self.loss_fn.forward(self.forward(x), y)
```

```
    def backward(self):
        grad = self.loss_fn.backward()
        for layer in reversed(self.layers):                                    # chain␣
↪rule, from the output back
            grad = layer.backward(grad)
        return grad                                                            # dL/
↪d(input)

    def linears(self):
        return [l for l in self.layers if isinstance(l, Linear)]

    def parameters(self):
        return [p for l in self.linears() for p in (l.W, l.b)]

    def gradients(self):
        return [g for l in self.linears() for g in (l.dW, l.db)]

    def sgd_step(self, lr):
        for l in self.linears():
            l.W -= lr * l.dW
            l.b -= lr * l.db
```

2.2 1(b)

The gradient calculations are compared with two independent references:

1. **Autograd.** The same network, with copies of the same parameters, is evaluated with ordinary PyTorch operations (`@`, `torch.relu`, `F.cross_entropy`) and differentiated with `loss.backward()`.
2. **Centred finite differences.** For a parameter entry θ_k: $\dfrac{\partial L}{\partial \theta_k} \approx \dfrac{L(\theta + he_k) - L(\theta - he_k)}{2h}$, with error $O(h^2)$.

The agreement is measured with the relative error $\mathrm{rel}(a, b) = \dfrac{\|a - b\|_2}{\|a\|_2 + \|b\|_2}$ (0 means identical, values near 1 mean unrelated) and with `torch.allclose`. The check runs in **float64** on a small MLP (3072–64–32–10) and 16 real CIFAR-10 images. Double precision is essential for finite differences, whose round-off error is about $\varepsilon_{\text{mach}}\,|L|/h$: with $h = 10^{-6}$ this is about 10^{-10} in float64 but would be about 10^{-1} in float32. The biases are set to random values so that their gradients are tested in a non-trivial state, and the gradient with respect to the input is checked too. Both activations are tested because Task 1(d) uses the sigmoid network.

Tolerances: autograd computes the same mathematics in a different order, so it must agree to round-off (`rtol=1e-7, atol=1e-10`). Finite differences are approximate, so a looser tolerance is used (`rtol=1e-5, atol=1e-7`).

[ ]:
```
def rel_error(a, b):
    """||a - b|| / (||a|| + ||b||)."""
```

```
    return ((a - b).norm() / (a.norm() + b.norm()).clamp_min(1e-300)).item()

def autograd_reference(net, x, y):
    """Run the same network with PyTorch autograd on copies of the same␣
 ↪parameters."""
    params = [p.clone().requires_grad_(True) for p in net.parameters()]
    x = x.clone().requires_grad_(True)
    h, k = x, 0
    for layer in net.layers:
        if isinstance(layer, Linear):
            h = h @ params[k] + params[k + 1]
            k += 2
        elif isinstance(layer, ReLU):
            h = torch.relu(h)
        else:
            h = torch.sigmoid(h)
    loss = F.cross_entropy(h, y)
    loss.backward()
    return loss.item(), [p.grad for p in params], x.grad

def finite_difference(net, x, y, tensor, n_checks=25, h=1e-6, gen=None):
    """Centred differences for n_checks random entries of `tensor` (perturbed␣
 ↪in place, then restored)."""
    flat = tensor.view(-1)
    idx = torch.randperm(flat.numel(), generator=gen)[:n_checks]
    numeric = torch.empty(len(idx), dtype=tensor.dtype)
    for k, i in enumerate(idx):
        old = flat[i].item()
        flat[i] = old + h; loss_plus = net.loss(x, y).item()
        flat[i] = old - h; loss_minus = net.loss(x, y).item()
        flat[i] = old
        numeric[k] = (loss_plus - loss_minus) / (2 * h)
    return idx, numeric

rows = []
fd_gen = torch.Generator().manual_seed(SEED)
for act in ['relu', 'sigmoid']:
    torch.manual_seed(SEED)
    net = MLP([3072, 64, 32, 10], activation=act, dtype=torch.float64,␣
 ↪device=torch.device('cpu'))
    for lin in net.linears():
        lin.b.normal_(0, 0.1)                                  # non-zero␣
 ↪biases
    x = X_train[:16].flatten(1).double().cpu()
    y = Y_train[:16].cpu()

    loss_mine = net.loss(x, y).item()                          # my forward pass
```

```
    dx_mine = net.backward()                                    # my backward␣
 ↪pass
    grads_mine = [g.clone() for g in net.gradients()] + [dx_mine]
    loss_ref, grads_ref, dx_ref = autograd_reference(net, x, y)
    grads_ref = grads_ref + [dx_ref]
    print(f'{act:>7s}: loss (mine) = {loss_mine:.12f}   loss (autograd) =␣
 ↪{loss_ref:.12f}')

    names = [f'{n}{i + 1}' for i in range(len(net.linears())) for n in ('W',␣
 ↪'b')] + ['input x']
    tensors = net.parameters() + [x]
    for name, t, g_mine, g_ref in zip(names, tensors, grads_mine, grads_ref):
        idx, g_num = finite_difference(net, x, y, t, gen=fd_gen)
        g_ana = g_mine.reshape(-1)[idx]
        rows.append({'activation': act, 'tensor': name, 'shape': str(tuple(t.
 ↪shape)),
                     'rel. err. vs autograd': rel_error(g_mine, g_ref),
                     'allclose (autograd)': torch.allclose(g_mine, g_ref,␣
 ↪rtol=1e-7, atol=1e-10),
                     'rel. err. vs finite diff.': rel_error(g_ana, g_num),
                     'allclose (finite diff.)': torch.allclose(g_ana, g_num,␣
 ↪rtol=1e-5, atol=1e-7)})

df_check = pd.DataFrame(rows)
for col in ['rel. err. vs autograd', 'rel. err. vs finite diff.']:
    df_check[col] = df_check[col].map(lambda v: f'{v:.2e}')
display(df_check)
print('All checks pass:', bool(df_check['allclose (autograd)'].all() and␣
 ↪df_check['allclose (finite diff.)'].all()))
```

```
   relu: loss (mine) = 2.797074244646   loss (autograd) = 2.797074244646
sigmoid: loss (mine) = 2.473725859027   loss (autograd) = 2.473725859027
```

	activation	tensor	shape	rel. err. vs autograd	allclose (autograd)	\
0	relu	W1	(3072, 64)	9.26e-17	True	
1	relu	b1	(64,)	6.84e-17	True	
2	relu	W2	(64, 32)	7.01e-17	True	
3	relu	b2	(32,)	8.52e-17	True	
4	relu	W3	(32, 10)	4.41e-17	True	
5	relu	b3	(10,)	4.50e-17	True	
6	relu	input x	(16, 3072)	9.40e-17	True	
7	sigmoid	W1	(3072, 64)	1.33e-16	True	
8	sigmoid	b1	(64,)	1.19e-16	True	
9	sigmoid	W2	(64, 32)	7.41e-17	True	
10	sigmoid	b2	(32,)	5.59e-17	True	
11	sigmoid	W3	(32, 10)	5.31e-17	True	
12	sigmoid	b3	(10,)	4.27e-17	True	
13	sigmoid	input x	(16, 3072)	1.54e-16	True	

	rel. err. vs finite diff.	allclose (finite diff.)
0	4.72e-09	True
1	2.10e-09	True
2	2.03e-09	True
3	1.08e-09	True
4	5.30e-10	True
5	6.42e-10	True
6	2.97e-08	True
7	3.54e-08	True
8	1.72e-08	True
9	8.67e-09	True
10	4.48e-09	True
11	1.44e-09	True
12	7.16e-10	True
13	5.24e-07	True

`All checks pass: True`

For both activations, every parameter tensor and the input gradient agree with autograd to a relative error between 4.3×10^{-17} and 1.5×10^{-16}, i.e. at the level of machine precision in float64 ($\varepsilon_{\text{mach}} \approx 1.1 \times 10^{-16}$), and `torch.allclose` returns `True` everywhere. Since autograd evaluates the same mathematical function, any remaining difference is only floating-point round-off from performing the operations in a different order. The two loss values also agree to all printed digits, which confirms that the forward passes are identical too.

The relative errors against centred finite differences are larger, between 5.3×10^{-10} and 5.2×10^{-7}, and `torch.allclose` is again `True` for every tensor. This is beacause the finite-difference estimate is itself approximate, with a truncation error of $O(h^2)$ and a round-off error of $O(\varepsilon_{\text{mach}}/h)$, both around 10^{-10} in absolute terms for $h = 10^{-6}$. A wrong backward pass, for example a missing transpose or a forgotten $1/N$, would give relative errors of order 10^{-1} to 1.

In the finite-difference calculation, errors are smallest for the last layer (W3, b3: about $5\text{–}7 \times 10^{-10}$) and grow towards the input, reaching 3.0×10^{-8} (ReLU) and 5.2×10^{-7} (sigmoid) for the input gradient. The absolute error of the finite-difference estimate is roughly constant, so the *relative* error is largest where the true gradients are smallest. Gradients become smaller towards the input, and much more so with the sigmoid, which is why the sigmoid network has the larger errors in W1, b1 and x. This is a sign of the vanishing gradients.

The finite-difference check is the more fundamental of the two because it does not depend on any other implementation of the same formulas. Together, the two checks show that the explicit backward passes of `Linear`, `ReLU`, `Sigmoid` and softmax cross-entropy are correct.

2.3 1(c) Training the MLP with SGD on flattened images

Each image is flattened to a vector of length $3 \cdot 32 \cdot 32 = 3072$. The network is 3072–512–256–10 with ReLU activations and Kaiming initialisation (1.71M parameters). It is trained with plain mini-batch SGD ($\theta \leftarrow \theta - \eta \nabla_\theta L$) with a constant learning rate $\eta = 0.05$, batch size 128 and 20 epochs.

[ ]:
```
seed_everything()
mlp = MLP([3072, 512, 256, 10], activation='relu', init='kaiming')
Xtr_flat, Xval_flat = X_train.flatten(1), X_val.flatten(1)          # (N, 3072) views, no copy
LR_MLP, BS_MLP, EPOCHS_MLP = 0.05, 128, 20
print('MLP parameters:', sum(p.numel() for p in mlp.parameters()))

def evaluate_mlp(net, X, Y, batch_size=1000):
    total_loss, correct = 0.0, 0
    for xb, yb in batches(X, Y, batch_size):
        logits = net.forward(xb)
        total_loss += net.loss_fn.forward(logits, yb).item() * len(yb)
        correct += (logits.argmax(1) == yb).sum().item()
    return total_loss / len(X), correct / len(X)

hist_mlp = dict(iter_loss=[], train_loss=[], train_acc=[], val_loss=[], val_acc=[],
                steps_per_epoch=len(Xtr_flat) // BS_MLP)
t0 = time.time()
for ep in range(EPOCHS_MLP):
    losses = []
    for xb, yb in batches(Xtr_flat, Y_train, BS_MLP, shuffle=True, drop_last=True):
        losses.append(mlp.loss(xb, yb))      # forward pass
        mlp.backward()                       # hand-written backward pass
        mlp.sgd_step(LR_MLP)                 # theta <- theta - lr * dL/dtheta
    hist_mlp['iter_loss'] += torch.stack(losses).cpu().tolist()
    tl, ta = evaluate_mlp(mlp, Xtr_flat, Y_train)
    vl, va = evaluate_mlp(mlp, Xval_flat, Y_val)
    for k, v in zip(['train_loss', 'train_acc', 'val_loss', 'val_acc'], [tl, ta, vl, va]):
        hist_mlp[k].append(v)
    print(f'epoch {ep + 1:2d} | train loss {tl:.4f} acc {100 * ta:5.2f}% | val loss {vl:.4f} acc {100 * va:5.2f}% | {time.time() - t0:5.1f}s')

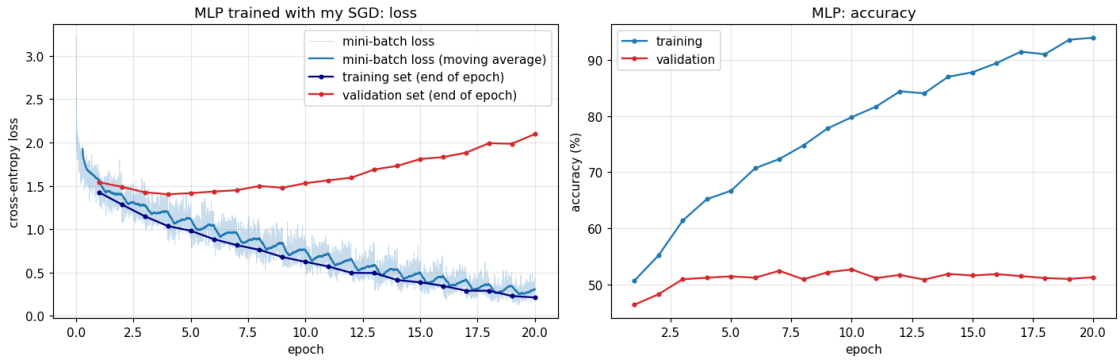
best_ep = int(np.argmax(hist_mlp['val_acc']))
print(f'\nFinal validation accuracy: {100 * hist_mlp["val_acc"][-1]:.2f}%   '
      f'(best {100 * hist_mlp["val_acc"][best_ep]:.2f}% at epoch {best_ep + 1})')
```

```
MLP parameters: 1707274
epoch  1 | train loss 1.4253 acc 50.64% | val loss 1.5417 acc 46.34% |   1.3s
epoch  2 | train loss 1.2835 acc 55.13% | val loss 1.4875 acc 48.22% |   1.8s
epoch  3 | train loss 1.1482 acc 61.33% | val loss 1.4253 acc 50.88% |   2.3s
epoch  4 | train loss 1.0360 acc 65.16% | val loss 1.4019 acc 51.16% |   2.8s
epoch  5 | train loss 0.9811 acc 66.66% | val loss 1.4155 acc 51.40% |   3.4s
epoch  6 | train loss 0.8848 acc 70.68% | val loss 1.4332 acc 51.16% |   3.9s
```

```
epoch  7 | train loss 0.8156 acc 72.32% | val loss 1.4490 acc 52.40% |   4.4s
epoch  8 | train loss 0.7607 acc 74.73% | val loss 1.4973 acc 50.88% |   4.9s
epoch  9 | train loss 0.6776 acc 77.78% | val loss 1.4774 acc 52.12% |   5.4s
epoch 10 | train loss 0.6222 acc 79.78% | val loss 1.5295 acc 52.64% |   5.9s
epoch 11 | train loss 0.5691 acc 81.65% | val loss 1.5633 acc 51.10% |   6.5s
epoch 12 | train loss 0.4953 acc 84.40% | val loss 1.5931 acc 51.66% |   7.0s
epoch 13 | train loss 0.4940 acc 84.04% | val loss 1.6883 acc 50.82% |   7.5s
epoch 14 | train loss 0.4123 acc 87.00% | val loss 1.7293 acc 51.84% |   8.0s
epoch 15 | train loss 0.3863 acc 87.79% | val loss 1.8088 acc 51.56% |   8.6s
epoch 16 | train loss 0.3457 acc 89.43% | val loss 1.8311 acc 51.80% |   9.2s
epoch 17 | train loss 0.2893 acc 91.46% | val loss 1.8822 acc 51.46% |   9.7s
epoch 18 | train loss 0.2893 acc 91.00% | val loss 1.9914 acc 51.10% |  10.2s
epoch 19 | train loss 0.2289 acc 93.58% | val loss 1.9852 acc 50.94% |  10.7s
epoch 20 | train loss 0.2112 acc 93.96% | val loss 2.0967 acc 51.24% |  11.2s

Final validation accuracy: 51.24%   (best 52.64% at epoch 10)
```

[ ]:
```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.3))
ax = axes[0]
it = np.arange(1, len(hist_mlp['iter_loss']) + 1) / hist_mlp['steps_per_epoch']
ax.plot(it, hist_mlp['iter_loss'], color='tab:blue', alpha=0.25, lw=0.6,␣
 ↪label='mini-batch loss')
plot_smoothed_loss(ax, hist_mlp, 'mini-batch loss (moving average)', k=100,␣
 ↪color='tab:blue')
ep_axis = np.arange(1, len(hist_mlp['val_loss']) + 1)
ax.plot(ep_axis, hist_mlp['train_loss'], 'o-', color='navy', ms=3,␣
 ↪label='training set (end of epoch)')
ax.plot(ep_axis, hist_mlp['val_loss'], 'o-', color='tab:red', ms=3,␣
 ↪label='validation set (end of epoch)')
ax.set_xlabel('epoch'); ax.set_ylabel('cross-entropy loss'); ax.set_title('MLP␣
 ↪trained with my SGD: loss'); ax.legend()
ax = axes[1]
ax.plot(ep_axis, 100 * np.array(hist_mlp['train_acc']), 'o-', ms=3,␣
 ↪label='training')
ax.plot(ep_axis, 100 * np.array(hist_mlp['val_acc']), 'o-', ms=3, color='tab:
 ↪red', label='validation')
ax.set_xlabel('epoch'); ax.set_ylabel('accuracy (%)'); ax.set_title('MLP:␣
 ↪accuracy'); ax.legend()
plt.tight_layout(); plt.show()
```



The mini-batch loss starts at about 3.2 (a little above ln 10 2.30, because Kaiming initialisation gives the untrained network fairly confident random predictions) and drops quickly during the first epoch. It is noisy because every update uses a different random batch of 128 images, while the end-of-epoch curves are smooth. The training loss then decreases steadily over all 20 epochs, from 1.43 to 0.21, and the training accuracy rises from 50.6% to 94.0%.

In the validation set, accuracy rises to about 51% by epoch 3 and then stays flat, between 50.8% and 52.6%: it peaks at **52.64% at epoch 10** and ends at **51.24%**. This is far above the random guessing(10%), so the network has learned something useful. The validation loss reaches its minimum of **1.40 at epoch 4** and then rises steadily to 2.10 at epoch 20.

This is clear **overfitting**. After about epoch 4 the extra training only improves the fit to the training images. The gap between training and validation accuracy grows from about 4 percentage points after epoch 1 to **42.7 pp** after epoch 20. The validation loss rises even though the validation accuracy stays flat. The network has become increasingly *over-confident*: its probabilities on training-like patterns get sharper, so the validation images it gets wrong are now wrong with high confidence, and cross-entropy penalises these heavily.

Two properties of the MLP explain both the low ceiling and the overfitting. First, flattening the image discards its spatial structure. Every pixel position has its own weights, so a pattern learned in one place (an edge, a wheel, an eye) does not transfer to other positions: there is no weight sharing and no translation equivariance. Second, the model is large relative to what it can usefully learn. Of its 1.71M parameters, 1.57M (92%) are in the first layer, which maps 3072 raw pixel values to 512 units. With no regularisation, this capacity is used to memorise the training images.

2.4 1(d) Per-layer gradient norms at initialisation in a 10-layer MLP: sigmoid vs ReLU

A 10-layer MLP (10 `Linear` layers, 3072–256–…–256–10) is created and, without any training, $\|\partial L/\partial W_\ell\|_F$ is computed for every layer ℓ with the hand-written backward pass, averaged over 10 mini-batches of 128 images. Layer 1 is next to the input and layer 10 produces the logits. Four settings are compared: Sigmoid and Relu activations with Xavier and Kaiming initialisation.

[ ]:

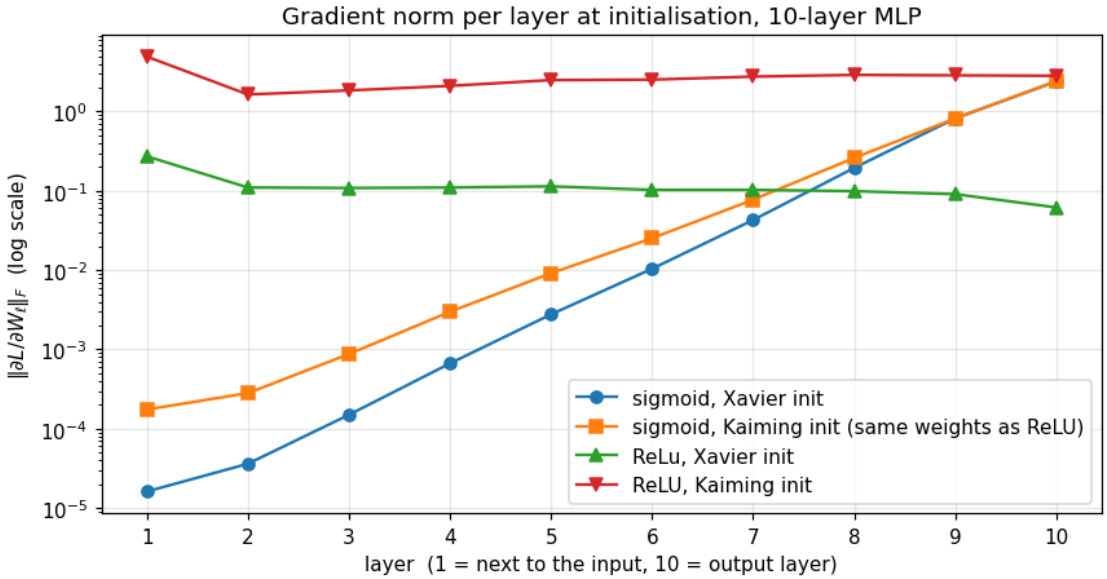
```
def layer_grad_norms(activation, init, depth=10, width=256, n_batches=10,␣
 ↪batch_size=128):
    """Mean Frobenius norm of dL/dW for each of the `depth` Linear layers of a␣
 ↪freshly initialised MLP."""
    seed_everything()
    net = MLP([3072] + [width] * (depth - 1) + [10], activation=activation,␣
 ↪init=init)
    norms = torch.zeros(depth)
    for k, (xb, yb) in enumerate(batches(Xtr_flat, Y_train, batch_size,␣
 ↪shuffle=True)):
        if k == n_batches:
            break
        net.loss(xb, yb)
        net.backward()
        norms += torch.stack([lin.dW.norm() for lin in net.linears()]).cpu()
    return (norms / n_batches).numpy()

configs_1d = {'sigmoid, Xavier init': ('sigmoid', 'xavier'),
              'sigmoid, Kaiming init (same weights as ReLU)': ('sigmoid',␣
 ↪'kaiming'),
              'ReLu, Xavier init': ('relu', 'xavier'),
              'ReLU, Kaiming init': ('relu', 'kaiming')}

grad_norms = {label: layer_grad_norms(*cfg) for label, cfg in configs_1d.
 ↪items()}

layers_axis = np.arange(1, 11)
fig, ax = plt.subplots(figsize=(8, 4.3))
for (label, g), m in zip(grad_norms.items(), ['o', 's', '^', 'v']):
    ax.semilogy(layers_axis, g, marker=m, label=label)
ax.set_xticks(layers_axis)
ax.set_xlabel('layer  (1 = next to the input, 10 = output layer)')
ax.set_ylabel(r'$\|\partial L / \partial W_\ell\|_F$  (log scale)')
ax.set_title('Gradient norm per layer at initialisation, 10-layer MLP'); ax.
 ↪legend()
plt.tight_layout(); plt.show()

df_1d = pd.DataFrame(grad_norms, index=[f'layer {i}' for i in layers_axis]).T
df_1d['layer 10 / layer 1'] = df_1d['layer 10'] / df_1d['layer 1']
display(df_1d.map(lambda v: f'{v:.2e}'))
```



	layer 1	layer 2	layer 3	\
sigmoid, Xavier init	1.61e-05	3.61e-05	1.50e-04	
sigmoid, Kaiming init (same weights as ReLU)	1.75e-04	2.81e-04	8.74e-04	
ReLu, Xavier init	2.72e-01	1.10e-01	1.09e-01	
ReLU, Kaiming init	4.93e+00	1.64e+00	1.85e+00	

	layer 4	layer 5	layer 6	\
sigmoid, Xavier init	6.67e-04	2.77e-03	1.04e-02	
sigmoid, Kaiming init (same weights as ReLU)	2.99e-03	9.14e-03	2.53e-02	
ReLu, Xavier init	1.10e-01	1.14e-01	1.03e-01	
ReLU, Kaiming init	2.11e+00	2.50e+00	2.53e+00	

	layer 7	layer 8	layer 9	\
sigmoid, Xavier init	4.29e-02	1.95e-01	8.22e-01	
sigmoid, Kaiming init (same weights as ReLU)	7.73e-02	2.60e-01	8.20e-01	
ReLu, Xavier init	1.03e-01	9.91e-02	9.09e-02	
ReLU, Kaiming init	2.76e+00	2.90e+00	2.86e+00	

	layer 10	layer 10 / layer 1
sigmoid, Xavier init	2.40e+00	1.49e+05
sigmoid, Kaiming init (same weights as ReLU)	2.43e+00	1.39e+04
ReLu, Xavier init	6.18e-02	2.27e-01
ReLU, Kaiming init	2.83e+00	5.73e-01

- **Sigmoid:** the gradient norm decays geometrically from the output to the input, so it is a straight line on the log plot. Layer 1 gets a gradient 1.5×10^5 times smaller than layer 10 with Xavier and 1.4×10^4 times smaller with Kaiming. This is about $3.8\times$ and $2.9\times$ per layer. This is the **vanishing gradient problem**.

- **ReLU with Kaiming:** the norm stays between 1.6 and 2.9 in layers 2 to 10, and the layer 10 / layer 1 ratio is 0.57. The gradient neither vanishes nor explodes.
- **ReLU with Xavier:** the norm is also flat across layers (about 0.1), but it is 15 to 45 times smaller than with Kaiming.

The backward pass multiplies the gradient by $W^\top$ and by $\phi'(z)$ at every layer, so each layer scales its norm by about $\sqrt{n\,\mathrm{Var}(W)\,\mathbb{E}[\phi'^2]}$, with width $n = 256$.

- **Sigmoid:** $\sigma'(z) \le 1/4$, so the factor is at most $\frac{1}{4}\sqrt{n\,\mathrm{Var}(W)}$. Xavier has $n\,\mathrm{Var}(W) \approx 1$, giving a shrinkage of $4\times$ per layer. Kaiming has $n\,\mathrm{Var}(W) = 2$, giving $4/\sqrt{2} \approx 2.8\times$. These match the measured $3.8\times$ and $2.9\times$. Kaiming is better here only because its weights are $\sqrt{2}$ larger, not because it suits the sigmoid. Xavier assumes an activation symmetric around 0, like tanh.
- **ReLU with Kaiming:** $\mathbb{E}[\phi'^2] = \frac{1}{2}$ and $n\,\mathrm{Var}(W) = 2$, so the factor is $\sqrt{2 \cdot \frac{1}{2}} = 1$. The signal is preserved in both directions.
- **ReLU with Xavier:** $n\,\mathrm{Var}(W) = 1$, so the factor is $1/\sqrt{2}$. The activations shrink by this factor going forward, and the backward gradient shrinks by it going towards the input. The weight gradient $\partial L/\partial W_\ell = a_{\ell-1}^\top \delta_\ell$ is the product of the two, so they cancel and each layer's gradient norm looks equal. The shrinkage is hidden, not absent: the overall level is lower by about $(1/\sqrt{2})^L$ for L layers, and for much deeper networks it would become tiny.
- **Layer 1:** its matrix is 3072×256, which has $12\times$ more entries than the hidden layers, so its Frobenius norm is larger. This explains the higher first point on the ReLU curves and the flatter start of the sigmoid curves.

Conclusion. In a deep sigmoid network, the early layers get gradients orders of magnitude smaller than the last layers, so they learn orders of magnitude more slowly. ReLU removes this layer-to-layer imbalance, and Kaiming initialisation also keeps the overall gradient scale constant. Initialisation cannot fully fix the sigmoid, whose derivative is at most $1/4$. Batch normalisation and the residual connections can address the problem in other ways.

3 Task 2 — Optimizers

3.1 2(a)

SGD with momentum (momentum μ, dampening τ):

$$\tilde{g}_t = g_t + \lambda\theta_{t-1}, \qquad b_t = \mu\, b_{t-1} + (1-\tau)\,\tilde{g}_t \;\; (b_1 = \tilde{g}_1), \qquad \theta_t = \theta_{t-1} - \eta\, b_t.$$

With Nesterov momentum the update is $\theta_t = \theta_{t-1} - \eta\,(\tilde{g}_t + \mu b_t)$, and $\mu = 0$ gives plain SGD. This form differs from the "classical" form $v_t = \mu v_{t-1} - \eta g_t$ only when the learning rate changes during training.

Adam :

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\tilde{g}_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)\tilde{g}_t^2,$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \theta_t = \theta_{t-1} - \eta\,\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Bias correction. Because $m_0 = v_0 = 0$, the moving averages are biased towards zero during the first steps: $\mathbb{E}[m_t] \approx (1-\beta_1^t)\,\mathbb{E}[g]$. Dividing by $1-\beta^t$ removes this bias. Without it the first updates would have the wrong size. With $\beta_1 = 0.9$ and $\beta_2 = 0.999$, the first step would be $\frac{(1-\beta_1)g}{\sqrt{1-\beta_2}\,|g|} \approx 3.2\,\eta$ instead of η, because v is much more strongly biased than m.

The implementation is written as $\theta \leftarrow \theta - \frac{\eta}{1-\beta_1^t} \cdot \frac{m_t}{\sqrt{v_t}/\sqrt{1-\beta_2^t}+\epsilon}$, which is algebraically identical to the formula above and is the same arrangement that PyTorch uses.

[ ]:
```
class SGDMomentum(torch.optim.Optimizer):
    """SGD with optional momentum, dampening, Nesterov momentum and L2 weight␣
↪decay (torch.optim.SGD conventions)."""
    def __init__(self, params, lr=1e-2, momentum=0.0, dampening=0.0,␣
↪weight_decay=0.0, nesterov=False):
        if lr < 0 or momentum < 0 or weight_decay < 0:
            raise ValueError('invalid hyper-parameter')
        if nesterov and (momentum <= 0 or dampening != 0):
            raise ValueError('Nesterov momentum requires momentum > 0 and zero␣
↪dampening')
        super().__init__(params, dict(lr=lr, momentum=momentum,␣
↪dampening=dampening,
                                      weight_decay=weight_decay,␣
↪nesterov=nesterov))

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
        for group in self.param_groups:
            lr, mu, damp = group['lr'], group['momentum'], group['dampening']
            wd, nesterov = group['weight_decay'], group['nesterov']
            for p in group['params']:
                if p.grad is None:
                    continue
                d_p = p.grad
                if wd != 0:
                    d_p = d_p.add(p, alpha=wd)                          # g +␣
↪lambda * theta
                if mu != 0:
                    state = self.state[p]
                    if 'momentum_buffer' not in state:                  # first␣
↪step: b_1 = g_1
                        buf = state['momentum_buffer'] = d_p.clone().detach()
                    else:
                        buf = state['momentum_buffer']
                        buf.mul_(mu).add_(d_p, alpha=1 - damp)          # b_t = mu␣
↪b_{t-1} + (1 - tau) g_t
                    d_p = d_p.add(buf, alpha=mu) if nesterov else buf
                p.add_(d_p, alpha=-lr)                                  # theta <-␣
↪theta - lr * update
```

```
        return loss


class Adam(torch.optim.Optimizer):
    """Adam with bias correction and optional L2 weight decay (torch.optim.Adam␣
 ↪conventions)."""
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999), eps=1e-8,␣
 ↪weight_decay=0.0):
        if lr < 0 or eps < 0 or not (0 <= betas[0] < 1 and 0 <= betas[1] < 1):
            raise ValueError('invalid hyper-parameter')
        super().__init__(params, dict(lr=lr, betas=betas, eps=eps,␣
 ↪weight_decay=weight_decay))

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
        for group in self.param_groups:
            lr, (beta1, beta2), eps, wd = group['lr'], group['betas'],␣
 ↪group['eps'], group['weight_decay']
            for p in group['params']:
                if p.grad is None:
                    continue
                g = p.grad
                if wd != 0:
                    g = g.add(p, alpha=wd)
                state = self.state[p]
                if len(state) == 0:
                    state['step'] = 0
                    state['exp_avg'] = torch.zeros_like(p, memory_format=torch.
 ↪preserve_format)      # m
                    state['exp_avg_sq'] = torch.zeros_like(p,␣
 ↪memory_format=torch.preserve_format)   # v
                state['step'] += 1
                t, m, v = state['step'], state['exp_avg'], state['exp_avg_sq']
                m.mul_(beta1).add_(g, alpha=1 - beta1)                # m_t = b1␣
 ↪m + (1 - b1) g
                v.mul_(beta2).addcmul_(g, g, value=1 - beta2)         # v_t = b2␣
 ↪v + (1 - b2) g^2
                bias_c1 = 1 - beta1 ** t                              # bias␣
 ↪corrections
                bias_c2 = 1 - beta2 ** t
                denom = (v.sqrt() / math.sqrt(bias_c2)).add_(eps)     #␣
 ↪sqrt(v_hat) + eps
```

```
                p.addcdiv_(m, denom, value=-lr / bias_c1)            # theta -=␣
↪lr * m_hat / (sqrt(v_hat) + eps)
        return loss
```

Showing that they match `torch.optim`. Two copies of a small CNN start from identical weights. One is trained with my optimizer and the other with the PyTorch optimizer, on exactly the same sequence of 100 mini-batches of real CIFAR-10 images. After 100 steps the parameters are compared with `torch.allclose`. Several configurations are tested (plain SGD, momentum, Nesterov with weight decay, dampening, and Adam with default and non-default settings). The comparison is done in float64 so that the only possible source of difference is round-off.

[ ]:
```
def compare_with_torch(make_mine, make_ref, n_steps=100, batch_size=16,␣
↪dtype=torch.float64):
    torch.manual_seed(SEED)
    model_mine = nn.Sequential(nn.Conv2d(3, 8, 3, padding=1), nn.ReLU(), nn.
↪MaxPool2d(4),
                               nn.Flatten(), nn.Linear(8 * 8 * 8, 10)).to(dtype)
    model_ref = copy.deepcopy(model_mine)                          # identical␣
↪starting weights
    init = [p.detach().clone() for p in model_mine.parameters()]
    opt_mine, opt_ref = make_mine(model_mine.parameters()), make_ref(model_ref.
↪parameters())
    X_cpu = X_train[:n_steps * batch_size].cpu().to(dtype)
    Y_cpu = Y_train[:n_steps * batch_size].cpu()
    for t in range(n_steps):
        xb, yb = X_cpu[t * batch_size:(t + 1) * batch_size], Y_cpu[t *␣
↪batch_size:(t + 1) * batch_size]
        for model, opt in ((model_mine, opt_mine), (model_ref, opt_ref)):
            opt.zero_grad()
            F.cross_entropy(model(xb), yb).backward()
            opt.step()
    pm, pr = list(model_mine.parameters()), list(model_ref.parameters())
    return {'max | _mine -  _torch|': max((a - b).abs().max().item() for a, b in␣
↪zip(pm, pr)),
            'max |Δ | from init': max((a - b).abs().max().item() for a, b in␣
↪zip(pm, init)),
            'torch.allclose': all(torch.allclose(a, b) for a, b in zip(pm, pr))}

OPT_TESTS = {
    'SGD (no momentum)':
        (lambda p: SGDMomentum(p, lr=0.05), lambda p: torch.optim.SGD(p, lr=0.
↪05)),
    'SGD, momentum 0.9':
        (lambda p: SGDMomentum(p, lr=0.05, momentum=0.9), lambda p: torch.optim.
↪SGD(p, lr=0.05, momentum=0.9)),
    'SGD, Nesterov 0.9, weight decay 5e-4':
```

```
        (lambda p: SGDMomentum(p, lr=0.05, momentum=0.9, nesterov=True,␣
 ↪weight_decay=5e-4),
         lambda p: torch.optim.SGD(p, lr=0.05, momentum=0.9, nesterov=True,␣
 ↪weight_decay=5e-4)),
    'SGD, momentum 0.8, dampening 0.1':
        (lambda p: SGDMomentum(p, lr=0.05, momentum=0.8, dampening=0.1),
         lambda p: torch.optim.SGD(p, lr=0.05, momentum=0.8, dampening=0.1)),
    'Adam (lr 1e-3, default betas)':
        (lambda p: Adam(p, lr=1e-3), lambda p: torch.optim.Adam(p, lr=1e-3)),
    'Adam (lr 3e-3, betas (0.8, 0.99), wd 1e-4)':
        (lambda p: Adam(p, lr=3e-3, betas=(0.8, 0.99), weight_decay=1e-4),
         lambda p: torch.optim.Adam(p, lr=3e-3, betas=(0.8, 0.99),␣
 ↪weight_decay=1e-4)),
}
df_opt = pd.DataFrame({name: compare_with_torch(*makers) for name, makers in␣
 ↪OPT_TESTS.items()}).T
for col in ['max |_mine - _torch|', 'max |Δ | from init']:
    df_opt[col] = df_opt[col].map(lambda v: f'{v:.2e}')
display(df_opt)
```

	max | _mine - _torch|	max |Δ | from init	torch.allclose
SGD (no momentum)	0.00e+00	1.22e-01	True
SGD, momentum 0.9	0.00e+00	5.81e+00	True
SGD, Nesterov 0.9, weight decay 5e-4	0.00e+00	6.99e+00	True
SGD, momentum 0.8, dampening 0.1	0.00e+00	1.09e+00	True
Adam (lr 1e-3, default betas)	5.55e-17	6.30e-02	True
Adam (lr 3e-3, betas (0.8, 0.99), wd 1e-4)	9.71e-17	1.41e-01	True

In all six configurations my optimizers reproduce `torch.optim` after 100 steps, and `torch.allclose` is `True`. For the four SGD variants (plain, momentum, Nesterov with weight decay, and dampening) the difference is exactly 0, so the results are bit-for-bit identical. This is expected as the implementationperforms the same floating-point operations in the same order as PyTorch's SGD. For Adam the maximum difference is 5.6×10^{-17} and 9.7×10^{-17}, which is at the level of float64 machine precision ($\approx 1.1 \times 10^{-16}$). These are round-off differences.

The match is not trivial, because the parameters have moved from their initial values. Same results over 100 steps also shows that the optimizer state is correct: the momentum buffers, Adam's m and v, and the step counter used in the bias correction. A mistake in any of these would accumulate over the steps and produce a visible difference. The non-default settings (Nesterov, dampening, weight decay, other betas) match too.

3.2 2(b)

The gradient is $\nabla f = (x,\ 50y)$ and the Hessian is $\mathrm{diag}(1, 50)$, so the curvature along y is 50 times larger than along x (condition number $\kappa = 50$). The level sets are long, narrow ellipses. All three methods start at $(-10, 1)$ and take 100 steps using my optimizer classes from 2(a). The gradient here is exact, so "SGD" is simply gradient descent. Plain SGD and SGD with momentum use the same learning rate $\eta = 0.035$, close to the stability limit $2/50 = 0.04$ set by the steep direction, and momentum uses $\mu = 0.9$. Adam uses $\eta = 0.3$, because its step size is set per coordinate, so its learning rate is on a different scale. Note that the two axes of the contour plot have different scales.

[ ]:
```
def f_quad(w):
    return 0.5 * (w[0] ** 2 + 50 * w[1] ** 2)

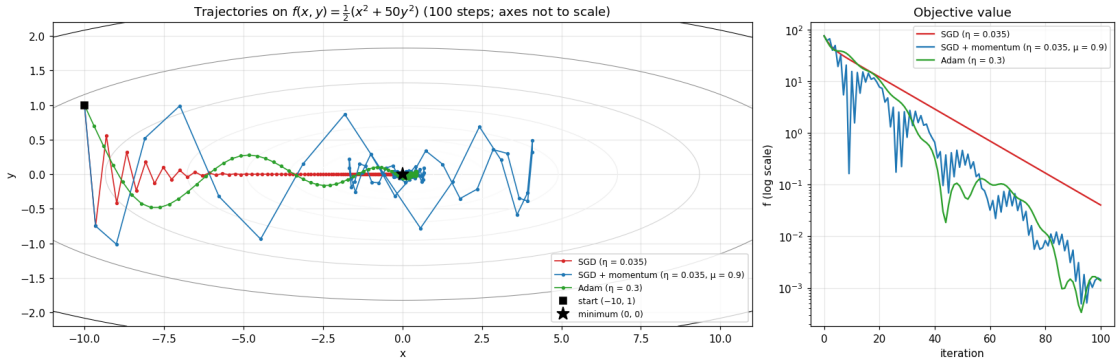
def trajectory(make_opt, n_steps=100, start=(-10.0, 1.0)):
    w = torch.tensor(start, dtype=torch.float64, requires_grad=True)
    opt = make_opt([w])
    path = [w.detach().clone()]
    for _ in range(n_steps):
        opt.zero_grad()
        f_quad(w).backward()              # gradient (x, 50 y)
        opt.step()
        path.append(w.detach().clone())
    return torch.stack(path).numpy()

TRAJ = {'SGD (  = 0.035)':                    lambda p: SGDMomentum(p, lr=0.035),
        'SGD + momentum (  = 0.035,   = 0.9)': lambda p: SGDMomentum(p, lr=0.
 ↪035, momentum=0.9),
        'Adam (  = 0.3)':                     lambda p: Adam(p, lr=0.3)}
paths = {name: trajectory(make) for name, make in TRAJ.items()}

xs, ys = np.meshgrid(np.linspace(-11, 11, 500), np.linspace(-2.2, 2.2, 500))
zs = 0.5 * (xs ** 2 + 50 * ys ** 2)
fig, axes = plt.subplots(1, 2, figsize=(15, 5), gridspec_kw={'width_ratios': [2.
 ↪3, 1]})
ax = axes[0]
cs = ax.contour(xs, ys, zs, levels=np.logspace(-2, 2.2, 16), cmap='Greys',␣
 ↪linewidths=0.7)
for (name, p), c in zip(paths.items(), ['tab:red', 'tab:blue', 'tab:green']):
    ax.plot(p[:, 0], p[:, 1], '-o', ms=2.5, lw=1.1, color=c, label=name)
ax.plot(-10, 1, 'ks', ms=7, label='start (-10, 1)')
ax.plot(0, 0, 'k*', ms=13, label='minimum (0, 0)')
ax.set_xlabel('x'); ax.set_ylabel('y'); ax.set_xlim(-11, 11); ax.set_ylim(-2.2,␣
 ↪2.2)
ax.set_title(r'Trajectories on $f(x,y)=\frac{1}{2}(x^2+50y^2)$ (100 steps; axes␣
 ↪not to scale)')
ax.legend(loc='lower right', fontsize=8)
```

```
ax = axes[1]
for (name, p), c in zip(paths.items(), ['tab:red', 'tab:blue', 'tab:green']):
    ax.semilogy(0.5 * (p[:, 0] ** 2 + 50 * p[:, 1] ** 2), color=c, label=name)
ax.set_xlabel('iteration'); ax.set_ylabel('f (log scale)'); ax.
 ↪set_title('Objective value'); ax.legend(fontsize=8)
plt.tight_layout(); plt.show()

for name, p in paths.items():
    k_x = int(np.argmax(p[:, 0]))
    print(f'{name:38s} final (x, y) = ({p[-1, 0]: .4f}, {p[-1, 1]: .2e})   f =␣
 ↪{f_quad(torch.tensor(p[-1])).item():.2e}'
          f'   largest x reached = {p[k_x, 0]:.2f} (iteration {k_x})')
```




```
SGD (  = 0.035)                              final (x, y) = (-0.2836,  3.21e-13)   f =
4.02e-02   largest x reached = -0.28 (iteration 100)
SGD + momentum (  = 0.035,   = 0.9)    final (x, y) = (-0.0451,  4.18e-03)   f =
1.45e-03   largest x reached = 4.09 (iteration 16)
Adam (  = 0.3)                               final (x, y) = (-0.0513, -1.64e-03)   f =
1.38e-03   largest x reached = 0.44 (iteration 60)
```

Because f is a sum of independent quadratics, plain gradient descent acts on each coordinate separately: $x_{t+1} = (1-\eta)\,x_t$ and $y_{t+1} = (1-50\eta)\,y_t$.

- **SGD.** With $\eta = 0.035$ the factor along y is $1 - 1.75 = -0.75$, so y changes sign at every step. The red path zig-zags across the narrow valley while the oscillation slowly dies out. Along the flat x direction the factor is 0.965, so progress is slow: after 100 steps $x \approx -10 \cdot 0.965^{100} \approx -0.28$ and $f \approx 4 \times 10^{-2}$. The learning rate cannot be increased to speed this up, because $\eta > 2/50$ would make the y iteration diverge. The steepest direction limits the step size, while the flattest direction determines how fast the method converges. This is the classic ill-conditioning problem.
- **SGD with momentum.** The velocity accumulates along x, where successive gradients have the same sign, and the effective step there grows towards $\eta/(1-\mu) = 10\eta$. The blue path therefore covers the flat direction much faster and ends at $f \approx 1.5 \times 10^{-3}$, about 25 times lower than SGD. The price is oscillation. With $\mu = 0.9$ the heavy-ball iteration is under-damped in *both* directions: its error contracts by only $\sqrt{\mu} \approx 0.95$ per step (against 0.75 for plain

SGD along y), so the path keeps bouncing across the valley in y for longer than plain SGD does. Along x it overshoots the minimum, reaching $x \approx +4.1$ at iteration 16, and then spirals in. The jagged, non-monotone objective curve on the right shows both effects. A smaller momentum ($\mu \approx 0.57$ is optimal for $\kappa = 50$) would damp the oscillations while keeping most of the speed-up.

- **Adam.** Adam divides each coordinate's step by a running estimate of that coordinate's gradient magnitude, so the first steps are about $\eta \cdot \mathrm{sign}(g)$ in each coordinate, whatever the curvature. The green path therefore starts out at about 45° in parameter space, ignoring the 50:1 difference in curvature. Adam is invariant to a rescaling of each coordinate, which is exactly the mismatch that hurts SGD here. Once y is near zero, x moves at almost a constant speed of about η per step, and Adam is close to the minimum after about 40 steps. Near the minimum the step does not shrink in proportion to the gradient, because $\hat{m}/\sqrt{\hat{v}}$ stays near ± 1 until v adapts. Adam therefore circles the optimum at a scale set by η and, with a constant learning rate, does not converge exactly: it ends at $f \approx 1.4 \times 10^{-3}$, similar to momentum. A decaying learning rate would remove this residual oscillation.

In short, momentum counters ill-conditioning by building up speed along the consistently-signed flat direction, and Adam counters it by rescaling each coordinate. Both end about 25–30 times lower in f than plain SGD after 100 steps; plain SGD would need about 150 steps to reach the same value.

3.3 2(c)

Network (`SmallCNN`): three stages of [conv3×3 → ReLU → conv3×3 → ReLU → max-pool 2×2] with 32, 64 and 128 channels, followed by a linear classifier on the flattened 128×4×4 feature map (about 0.31M parameters). The conv layers use Kaiming initialisation and the classifier keeps PyTorch's default initialisation, which gives small initial logits. Batch normalisation is deliberately left out here, because it makes networks less sensitive to the learning rate and would hide the differences between the optimizers.

Protocol. 10 epochs on the 45k training images, batch size 128, a constant learning rate and no augmentation or regularisation. Every run starts from the same initial weights and sees the same sequence of batches (the seed is reset before each run), so the optimizer and the learning rate are the only differences. The learning rates are spaced by factors of 10: $\{10^{-3}, 10^{-2}, 10^{-1}\}$ for SGD and SGD with momentum ($\mu = 0.9$), and $\{10^{-4}, 10^{-3}, 10^{-2}\}$ for Adam, whose usual range is about ten times lower. The validation accuracy after the last epoch is reported.

[ ]:
```
class SmallCNN(nn.Module):
    """[conv-(BN)-ReLU-conv-(BN)-ReLU-maxpool-(dropout)] x 3 with width,␣
↪2*width, 4*width channels, then a linear layer."""
    def __init__(self, width=32, batch_norm=False, dropout=0.0, num_classes=10):
        super().__init__()
        chans = [3, width, 2 * width, 4 * width]
        layers = []
        for i in range(3):
            for j in range(2):
                c_in = chans[i] if j == 0 else chans[i + 1]
                layers.append(nn.Conv2d(c_in, chans[i + 1], 3, padding=1,␣
↪bias=not batch_norm))
```

```
                if batch_norm:
                    layers.append(nn.BatchNorm2d(chans[i + 1]))
                layers.append(nn.ReLU(inplace=True))
            layers.append(nn.MaxPool2d(2))
            if dropout > 0:
                layers.append(nn.Dropout(dropout))
        self.features = nn.Sequential(*layers)
        self.classifier = nn.Linear(chans[3] * 4 * 4, num_classes)
        for m in self.modules():   # Kaiming init for the convs; the linear␣
 ↪classifier keeps PyTorch's default (smaller) init
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, nonlinearity='relu')
                if m.bias is not None:
                    nn.init.zeros_(m.bias)

    def forward(self, x):
        return self.classifier(torch.flatten(self.features(x), 1))

print('SmallCNN parameters (no BN):', f'{count_params(SmallCNN()):,}')
```

```
SmallCNN parameters (no BN): 307,498
```

[ ]:
```
OPT_GRID = {
    'SGD':          ([1e-3, 1e-2, 1e-1], lambda params, lr: SGDMomentum(params,␣
 ↪lr=lr)),
    'SGD+momentum': ([1e-3, 1e-2, 1e-1], lambda params, lr: SGDMomentum(params,␣
 ↪lr=lr, momentum=0.9)),
    'Adam':         ([1e-4, 1e-3, 1e-2], lambda params, lr: Adam(params,␣
 ↪lr=lr)),
}
EPOCHS_2C, BS_2C = 10, 128

results_2c = {}
for opt_name, (lrs, make_opt) in OPT_GRID.items():
    for lr in lrs:
        run_name = f'{opt_name} lr={lr:g}'
        def run(make_opt=make_opt, lr=lr, run_name=run_name):
            seed_everything()                    # same initial weights and␣
 ↪batch order for every run
            model = SmallCNN(batch_norm=False).to(device)
            return train(model, make_opt(model.parameters(), lr), EPOCHS_2C,␣
 ↪X_train, Y_train, X_val, Y_val,
                         batch_size=BS_2C, name=run_name)
        results_2c[(opt_name, lr)] = cached('task2c_' + run_name, run)
```

[ ]:
```
# ---- table: final validation accuracy for every (optimizer, learning rate)␣
 ↪----
```

```
rows = []
for (opt_name, lr), h in results_2c.items():
    rows.append({'optimizer': opt_name, 'learning rate': lr, 'final val acc␣
 ↪(%)': 100 * h['val_acc'][-1],
                 'best val acc (%)': 100 * max(h['val_acc']), 'final train␣
 ↪loss': h['train_loss'][-1],
                 'diverged': h['diverged']})
df_2c = pd.DataFrame(rows)
display(df_2c.round(3).assign(**{'learning rate': df_2c['learning rate'].
 ↪map(lambda v: f'{v:g}')}))
best_lr = {o: float(df_2c[df_2c.optimizer == o].sort_values('final val acc␣
 ↪(%)').iloc[-1]['learning rate']) for o in OPT_GRID}
print('best learning rate per optimizer:', best_lr)

colors_2c = {'SGD': 'tab:red', 'SGD+momentum': 'tab:blue', 'Adam': 'tab:green'}
fig, axes = plt.subplots(1, 3, figsize=(17, 4.5))
ax = axes[0]
for o, (lrs, _) in OPT_GRID.items():
    accs = [100 * results_2c[(o, lr)]['val_acc'][-1] for lr in lrs]
    ax.semilogx(lrs, accs, 'o-', color=colors_2c[o], label=o, ms=6)
    for lr, a in zip(lrs, accs):
        if results_2c[(o, lr)]['diverged']:
            ax.annotate('diverged', (lr, a), textcoords='offset points',␣
 ↪xytext=(-20, 8), fontsize=8, color=colors_2c[o])
ax.axhline(10, color='grey', ls=':', lw=1, label='chance (10%)')
ax.set_xlabel('learning rate (log scale)'); ax.set_ylabel('validation accuracy␣
 ↪after 10 epochs (%)')
ax.set_title('Learning-rate sensitivity'); ax.legend()
ax = axes[1]
for o in OPT_GRID:
    plot_smoothed_loss(ax, results_2c[(o, best_lr[o])], f'{o} (lr={best_lr[o]:
 ↪g})', color=colors_2c[o])
ax.set_xlabel('epoch'); ax.set_ylabel('training loss (moving average)'); ax.
 ↪set_title('Training loss at the best learning rate'); ax.legend()
ax = axes[2]
for o in OPT_GRID:
    h = results_2c[(o, best_lr[o])]
    ax.plot(np.arange(1, len(h['val_acc']) + 1), 100 * np.array(h['val_acc']),␣
 ↪'o-', ms=3, color=colors_2c[o],
            label=f'{o} (lr={best_lr[o]:g})')
ax.set_xlabel('epoch'); ax.set_ylabel('validation accuracy (%)'); ax.
 ↪set_title('Validation accuracy at the best learning rate'); ax.legend()
plt.tight_layout(); plt.show()
```

```
      optimizer learning rate  final val acc (%)  best val acc (%)  \
0           SGD         0.001              50.20             50.20
1           SGD          0.01              63.44             64.38
```

```
2            SGD          0.1              70.04              71.28
3  SGD+momentum        0.001              68.74              68.74
4  SGD+momentum         0.01              73.78              75.36
5  SGD+momentum          0.1               9.42              10.14
6           Adam       0.0001              66.28              66.74
7           Adam        0.001              74.24              75.86
8           Adam         0.01              54.18              54.44

   final train loss  diverged
0             1.369     False
1             0.821     False
2             0.583     False
3             0.754     False
4             0.300     False
5             2.305     False
6             0.776     False
7             0.181     False
8             1.272     False

best learning rate per optimizer: {'SGD': 0.1, 'SGD+momentum': 0.01, 'Adam':
0.001}
```



The best learning rates are $\eta = 0.1$ for SGD (70.04%), $\eta = 0.01$ for SGD with momentum (73.78%) and $\eta = 10^{-3}$ for Adam (74.24%). Momentum and Adam are effectively tied: their 0.5 pp difference is within the noise of a single run on 5,000 validation images (standard error about 0.6 pp). Plain SGD is clearly behind, by about 4 pp.

- **SGD** improves steadily with the learning rate (50.2% $\rightarrow$ 63.4% $\rightarrow$ 70.0%), because with a small step 10 epochs are not enough to train. Its best value is at the edge of the grid, so a larger rate might do even better.
- **SGD with momentum** behaves like SGD with a learning rate about 10$\times$ larger, because the velocity accumulates about $1/(1-\mu) = 10$ gradients. Its curve is therefore shifted one decade to the left: $\eta = 10^{-3}$ and 10^{-2} give 68.7% and 73.8%, compared with 63.4% and 70.0% for SGD at 10^{-2} and 10^{-1}. At $\eta = 0.1$ the effective step (about 1) is far too large for a network without batch normalisation, and training collapses. Accuracy stays at chance (9.4%) and the loss stays at 2.305, which is $\ln 10$, so the network predicts the same probability for every class. The `diverged` flag is `False` only because the loss stayed finite and never became NaN.
- **Adam** works best at its default 10^{-3}. At 10^{-4} the steps are too small for 10 epochs (66.3%,

still improving). At 10^{-2} it reaches only 54.2% and its training loss stalls at about 1.27. Adam moves each weight by roughly η per step whatever the gradient scale, and 0.01 is large compared with the weights in most layers (initial standard deviation of about 0.01–0.08), so the iterates keep jumping around instead of settling.

Training loss at the best rates: Adam lowers the training loss fastest (epoch-1 mean 1.50, against 1.57 for momentum and 2.09 for SGD) and ends lowest (0.18, against 0.30 and 0.58). SGD is the slowest even at its best rate. A lower training loss does not mean higher validation accuracy, though. For Adam and momentum the validation loss is lowest at epoch 5–7 (about 0.76) and then rises (to 1.04 and 0.92 at epoch 10) while the training loss keeps falling, so they begin to overfit, as no regularisation is used. Their best validation accuracies (75.9% and 75.4%) occur before the last epoch.

Sensitivity: Every optimiser works well only within about one decade of learning rates, and the windows sit in different places: near 10^{-3} for Adam, near 10^{-2} for momentum, and at or above 10^{-1} for SGD. The failure modes also differ. SGD and Adam degrade gradually. Adam was the easiest to get right, because its default value worked without tuning and its step size does not depend on the gradient scale. SGD-type methods need the learning rate matched to the gradient scale. These are 10-epoch runs with a constant learning rate and a single seed, so they measure speed of learning as much as final quality. The slowest settings (SGD at 10^{-3}, Adam at 10^{-4}) are under-trained rather than bad.

4 Task 3 - Regularisation and residual connections

4.1 3(a)

The `SmallCNN` from 2(c), now **with batch normalisation**, is trained on only 5,000 training images (the first 5k of the already random 45k split), which makes overfitting severe. The full 5k validation set is used for evaluation. Seven settings are compared:

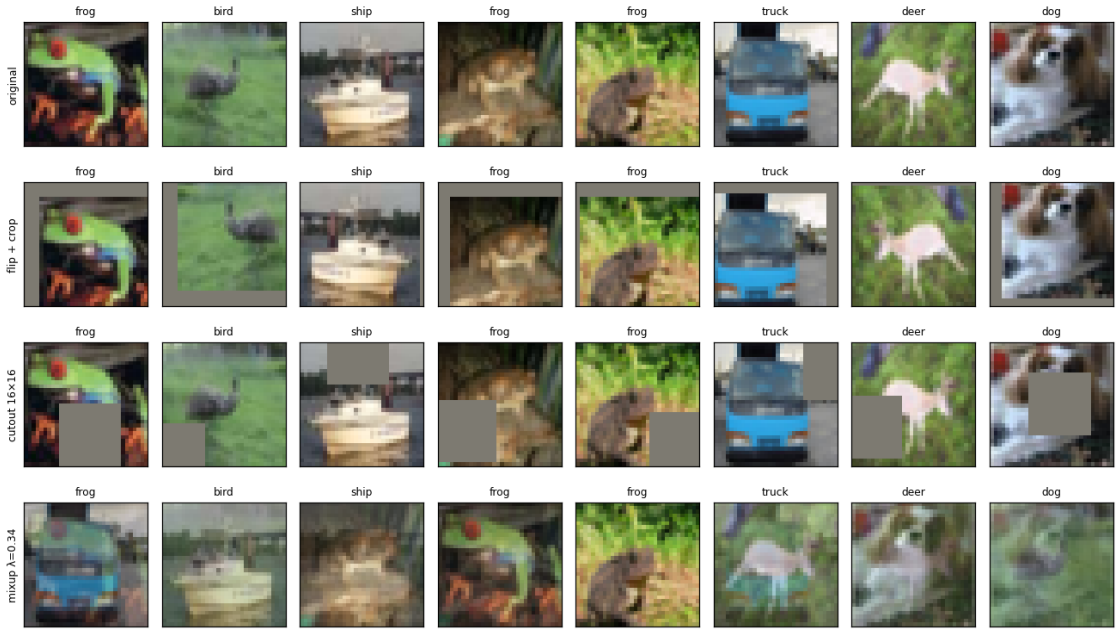
setting	what it does
baseline	no regularisation
weight decay	L2 penalty $\lambda = 5 \times 10^{-4}$, i.e. $\lambda\theta$ is added to every gradient
dropout	`nn.Dropout(p=0.3)` after each of the three pooling stages
flip + crop	random horizontal flip, then a random 32×32 crop from the image zero-padded by 4 pixels
cutout	a random 16×16 square is set to zero (the mean colour) in every image
mixup	train on $\lambda x_i + (1-\lambda)x_j$ with loss $\lambda\,\ell(y_i) + (1-\lambda)\,\ell(y_j)$, where $\lambda \sim \mathrm{Beta}(0.4, 0.4)$
all combined	all of the above together

Cutout and mixup are implemented in the utilities section (`cutout`, `mixup`).

Training: SGD with Nesterov momentum 0.9, learning rate 0.05 with a 3-epoch linear warm-up and cosine decay, batch size 128, 60 epochs, identical initialisation for every setting. Train accuracy is measured on the 5k training images in evaluation mode at the end of each epoch.

First, a visual check that the augmentations do what they should:

[ ]:
```
seed_everything(0)
sample, labels = X_train[:8], Y_train[:8]
x_mix, y_a, y_b, lam = mixup(sample, labels, alpha=0.4)
show_rows([('original', sample, labels),
           ('flip + crop', random_crop(random_flip(sample)), labels),
           ('cutout 16x16', cutout(sample, 16), labels),
           (f'mixup λ={lam:.2f}', x_mix, y_a)], n=8)
```



[ ]:
```
N_SMALL = 5000
X_small, Y_small = X_train[:N_SMALL], Y_train[:N_SMALL]      # the split is␣
 ↪already random -> random 5k subset
EPOCHS_3A, BS_3A, LR_3A = 60, 128, 0.05
WD, P_DROP, CUTOUT_SIZE, MIXUP_ALPHA = 5e-4, 0.3, 16, 0.4

REG_CONFIGS = {
    'baseline':     dict(),
    'weight decay': dict(weight_decay=WD),
    'dropout':      dict(dropout=P_DROP),
    'flip + crop':  dict(flip_crop=True),
    'cutout':       dict(cutout=CUTOUT_SIZE),
    'mixup':        dict(mixup=MIXUP_ALPHA),
    'all combined': dict(weight_decay=WD, dropout=P_DROP, flip_crop=True,␣
 ↪cutout=CUTOUT_SIZE, mixup=MIXUP_ALPHA),
}
```

```
print('SmallCNN with BN:', f'{count_params(SmallCNN(batch_norm=True)):,}',␣
 ↪'parameters')

results_3a = {}
for name, cfg in REG_CONFIGS.items():
    def run(cfg=cfg, name=name):
        seed_everything()
        model = SmallCNN(batch_norm=True, dropout=cfg.get('dropout', 0.0)).
 ↪to(device)
        opt = torch.optim.SGD(model.parameters(), lr=LR_3A, momentum=0.9,␣
 ↪nesterov=True,
                              weight_decay=cfg.get('weight_decay', 0.0))
        return train(model, opt, EPOCHS_3A, X_small, Y_small, X_val, Y_val,␣
 ↪batch_size=BS_3A,
                     augment=make_augment(cfg.get('flip_crop', False), cfg.
 ↪get('cutout', 0)),
                     mixup_alpha=cfg.get('mixup', 0.0),␣
 ↪lr_schedule=cosine_schedule(warmup_frac=0.05),
                     eval_train=(X_small, Y_small), name=name)
    results_3a[name] = cached('task3a_' + name, run)
```

[ ]:
```
rows = []
for name, h in results_3a.items():
    tr, va = 100 * h['train_acc'][-1], 100 * h['val_acc'][-1]
    rows.append({'setting': name, 'train acc (%)': tr, 'val acc (%)': va, 'gap␣
 ↪(pp)': tr - va,
                 'best val acc (%)': 100 * max(h['val_acc']), 'final val loss':␣
 ↪h['val_loss'][-1]})
df_3a = pd.DataFrame(rows).set_index('setting')
display(df_3a.round(2))

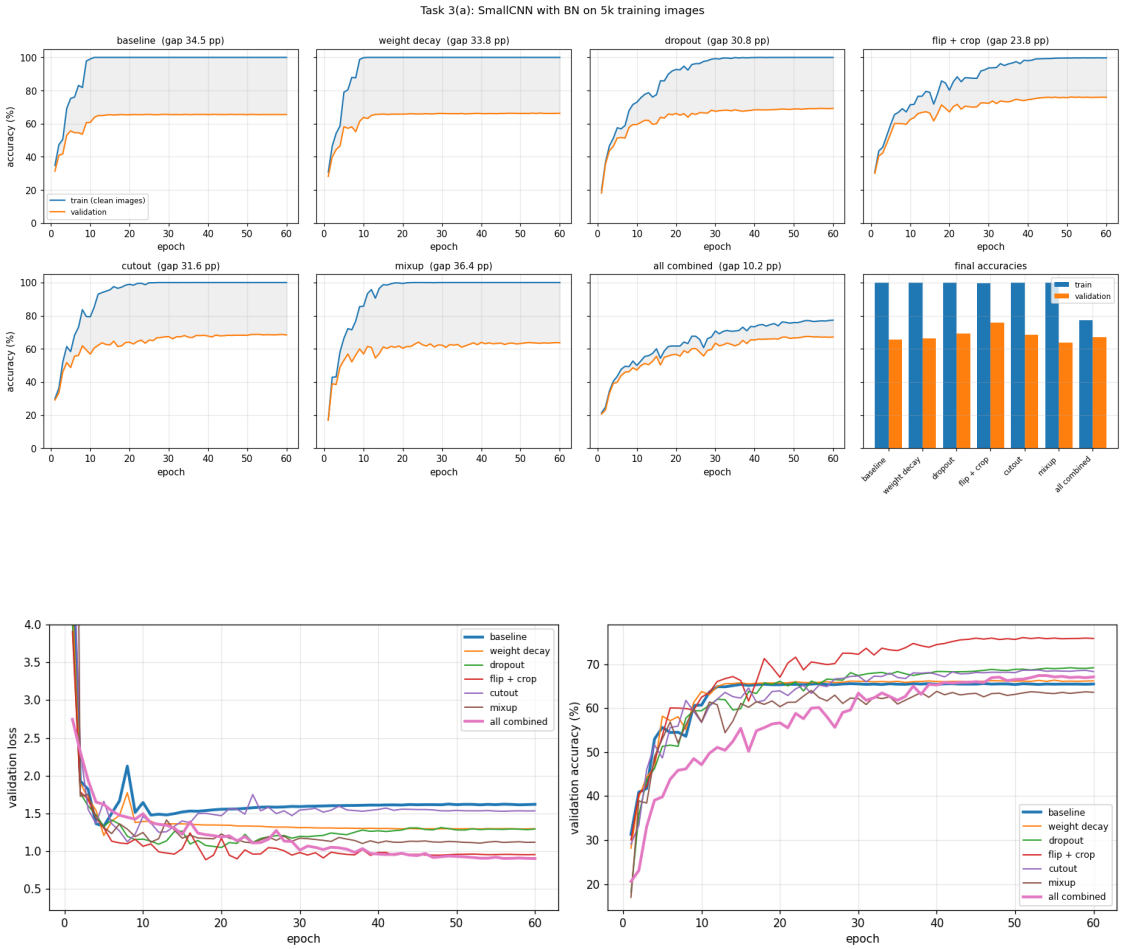
fig, axes = plt.subplots(2, 4, figsize=(17, 7.5), sharey=True)
for ax, (name, h) in zip(axes.flat, results_3a.items()):
    ep = np.arange(1, len(h['val_acc']) + 1)
    ax.plot(ep, 100 * np.array(h['train_acc']), label='train (clean images)')
    ax.plot(ep, 100 * np.array(h['val_acc']), label='validation')
    ax.fill_between(ep, 100 * np.array(h['val_acc']), 100 * np.
 ↪array(h['train_acc']), alpha=0.12, color='grey')
    ax.set_title(f'{name}  (gap {df_3a.loc[name, "gap (pp)"]:.1f} pp)',␣
 ↪fontsize=10)
    ax.set_xlabel('epoch')
for ax in axes[:, 0]:
    ax.set_ylabel('accuracy (%)')
axes[0, 0].legend(fontsize=8)
ax = axes.flat[-1]
pos = np.arange(len(df_3a))
```

```
ax.bar(pos - 0.2, df_3a['train acc (%)'], 0.4, label='train')
ax.bar(pos + 0.2, df_3a['val acc (%)'], 0.4, label='validation')
ax.set_xticks(pos); ax.set_xticklabels(df_3a.index, rotation=45, ha='right',␣
  ↪fontsize=8)
ax.set_title('final accuracies', fontsize=10); ax.legend(fontsize=8)
plt.suptitle('Task 3(a): SmallCNN with BN on 5k training images', y=1.0)
plt.tight_layout(); plt.show()

fig, axes = plt.subplots(1, 2, figsize=(14, 4.3))
for name, h in results_3a.items():
    ep = np.arange(1, len(h['val_acc']) + 1)
    lw = 2.5 if name in ('baseline', 'all combined') else 1.2
    axes[0].plot(ep, h['val_loss'], lw=lw, label=name)
    axes[1].plot(ep, 100 * np.array(h['val_acc']), lw=lw, label=name)
axes[0].set_ylabel('validation loss'); axes[1].set_ylabel('validation accuracy␣
  ↪(%)')
for ax in axes:
    ax.set_xlabel('epoch'); ax.legend(fontsize=8)
axes[0].set_ylim(top=min(4.0, axes[0].get_ylim()[1]))
plt.tight_layout(); plt.show()
```

```
              train acc (%)  val acc (%)  gap (pp)  best val acc (%)  \
setting
baseline             100.00        65.52     34.48             65.62
weight decay         100.00        66.24     33.76             66.40
dropout               99.98        69.22     30.76             69.22
flip + crop           99.74        75.90     23.84             76.10
cutout               100.00        68.36     31.64             68.70
mixup                100.00        63.64     36.36             64.04
all combined          77.30        67.14     10.16             67.44

              final val loss
setting
baseline                1.62
weight decay            1.30
dropout                 1.29
flip + crop             0.96
cutout                  1.53
mixup                   1.12
all combined            0.90
```



With only 5k images the baseline memorises the training set (100% train accuracy by epoch 12) but reaches only 65.5% on validation, a gap of 34.5 pp. Its validation loss rises from 1.49 at epoch 12 to 1.62 at epoch 60 while the accuracy stays flat, which is classic overfitting. Ranked by validation accuracy: flip + crop (75.9%) > dropout (69.2%) > cutout (68.4%) > all combined (67.1%) > weight decay (66.2%) > baseline (65.5%) > mixup (63.6%).

- **Flip + crop** is by far the best single method (+10.4 pp, validation loss 0.96). It encodes invariances that really hold for natural images, which effectively enlarges the training set. The network still fits the clean training images (99.7%), so the gap shrinks mainly because validation accuracy rises.
- **Dropout** (+3.7 pp) slows memorisation (99% train accuracy only at epoch 30, against epoch 12 for the baseline) but adds no new information about the data.
- **Cutout** (+2.8 pp) helps less than flip + crop. It hides parts of the image but does not create new views, and the network still reaches 100% train accuracy.
- **Weight decay** (+0.7 pp) is within single-run noise for accuracy (one standard error is about 0.7 pp on 5,000 images), but it lowers the final validation loss from 1.62 to 1.30. This most likely comes from limiting the size of the logits, so predictions are less over-confident, rather than from changing which images are classified correctly.

- **Mixup** lowers the validation loss (1.12 vs 1.62) but not the error rate (−1.9 pp). It reduces over-confidence, not mistakes. Its training loss stays near 0.6 because mixed targets cannot be fitted exactly, yet the network still memorises the clean images (100%). Mixup typically pays off with longer training or larger models, which this setting does not provide.
- **All combined** has the smallest gap (10.2 pp) and the lowest validation loss (0.90), but its accuracy (67.1%) is below flip + crop alone. Its train accuracy is only 77% and it was still improving through epoch 54 (validation 55.6% at epoch 18, 67.4% at epoch 54), so it **under-fits**: the combined regularisation is too strong for this model and 60-epoch budget.

The most effective regulariser was the one that encodes a true property of the data (flip + crop), and stacking every regulariser did worse than that one alone.

4.2 3(b)

Architecture: a 3×3 conv with 16 channels, then three stages of n *basic blocks* with 16, 32 and 64 channels on 32×32, 16×16 and 8×8 feature maps, then global average pooling and a linear layer, for $2 + 6n$ weighted layers. A basic block is conv3×3 → BN → ReLU → conv3×3 → BN, then ReLU (Resnet has skip connection before the activation). The first block of stages 2 and 3 halves the resolution with stride 2. All conv weights use Kaiming (He) normal initialization, BN weights and biases start at 1 and 0, and the final linear layer keeps PyTorch's default initialization.

Training: 20 epochs on the 45k training images with flip + crop, SGD with momentum 0.9, learning rate 0.1 (with a 1-epoch warm-up and cosine decay), weight decay 10^{-4}, batch size 128. These are the settings of the original paper, apart from the shortened schedule.

[16]:
```
class BasicBlock(nn.Module):
    """conv3x3-BN-ReLU-conv3x3-BN (+ identity shortcut) -ReLU.  With␣
↪shortcut=False the block is 'plain'."""
    def __init__(self, c_in, c_out, stride=1, shortcut=True):
        super().__init__()
        self.conv1 = nn.Conv2d(c_in, c_out, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(c_out)
        self.conv2 = nn.Conv2d(c_out, c_out, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(c_out)
        self.shortcut, self.stride, self.extra = shortcut, stride, c_out - c_in

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        if self.shortcut:
            sc = x
            if self.stride != 1 or self.extra != 0:          # option A:␣
↪subsample + zero-pad channels (no parameters)
                sc = x[:, :, ::self.stride, ::self.stride]
                sc = F.pad(sc, (0, 0, 0, 0, self.extra // 2, self.extra - self.
↪extra // 2))
            out = out + sc
        return F.relu(out)
```

```
class CifarResNet(nn.Module):
    """CIFAR-style network: conv3x3 stem, three stages of BasicBlocks, global␣
 ↪average pooling, linear layer."""
    def __init__(self, blocks=(3, 3, 3), widths=(16, 32, 64), shortcut=True,␣
 ↪num_classes=10):
        super().__init__()
        self.stem = nn.Sequential(nn.Conv2d(3, widths[0], 3, 1, 1, bias=False),␣
 ↪nn.BatchNorm2d(widths[0]), nn.ReLU(inplace=True))
        layers, c_in = [], widths[0]
        for stage, (n, w) in enumerate(zip(blocks, widths)):
            for b in range(n):
                stride = 2 if (stage > 0 and b == 0) else 1      # downsample␣
 ↪at the start of stages 2 and 3
                layers.append(BasicBlock(c_in, w, stride, shortcut))
                c_in = w
        self.layers = nn.Sequential(*layers)
        self.fc = nn.Linear(c_in, num_classes)
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, nonlinearity='relu')
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.ones_(m.weight); nn.init.zeros_(m.bias)
        # the final linear layer keeps PyTorch's default initialisation (small␣
 ↪initial logits)

    def forward(self, x):
        x = self.layers(self.stem(x))
        return self.fc(F.adaptive_avg_pool2d(x, 1).flatten(1))   # global␣
 ↪average pooling


def cifar_net(depth, shortcut):
    assert (depth - 2) % 6 == 0, 'depth must be 6n + 2'
    n = (depth - 2) // 6
    return CifarResNet((n, n, n), (16, 32, 64), shortcut)

DEPTHS = [8, 20, 56]
df_params = pd.DataFrame([{'depth': d, 'n (blocks per stage)': (d - 2) // 6,
                           'plain: parameters': count_params(cifar_net(d,␣
 ↪False)),
                           'residual: parameters': count_params(cifar_net(d,␣
 ↪True))} for d in DEPTHS]).set_index('depth')
display(df_params)
```

depth	n (blocks per stage)	plain: parameters	residual: parameters

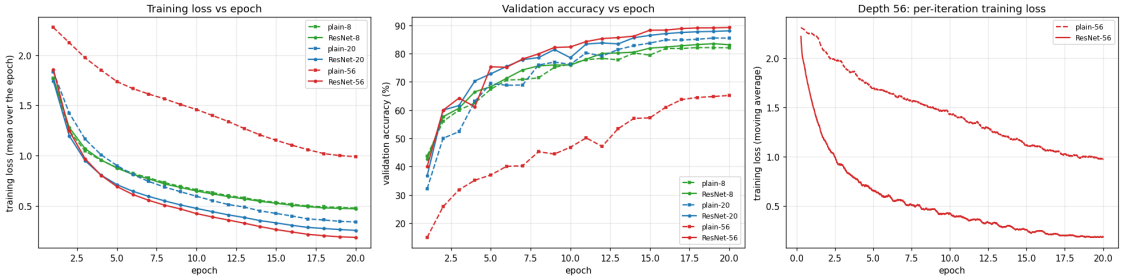
8	1	75290	75290
20	3	269722	269722
56	9	853018	853018

[ ]:
```
EPOCHS_3B = 20
results_3b = {}
for depth in DEPTHS:
    for shortcut in [False, True]:
        run_name = f'{"ResNet" if shortcut else "plain"}-{depth}'
        def run(depth=depth, shortcut=shortcut, run_name=run_name):
            seed_everything()
            model = cifar_net(depth, shortcut).to(device)
            opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9,␣
 ↪weight_decay=1e-4)
            return train(model, opt, EPOCHS_3B, X_train, Y_train, X_val, Y_val,␣
 ↪batch_size=128,
                         augment=make_augment(flip_crop=True),␣
 ↪lr_schedule=cosine_schedule(warmup_frac=0.05),
                         name=run_name)
        results_3b[(depth, shortcut)] = cached('task3b_' + run_name, run)
```

[ ]:
```
colors_3b = {8: 'tab:green', 20: 'tab:blue', 56: 'tab:red'}
fig, axes = plt.subplots(1, 3, figsize=(18, 4.6))
for (depth, shortcut), h in results_3b.items():
    ep = np.arange(1, len(h['train_loss']) + 1)
    style = dict(ls='-' if shortcut else '--', marker='o' if shortcut else 's',␣
 ↪ms=3, color=colors_3b[depth],
                 label=f'{"ResNet" if shortcut else "plain"}-{depth}')
    axes[0].plot(ep, h['train_loss'], **style)
    axes[1].plot(ep, 100 * np.array(h['val_acc']), **style)
    if depth == 56:
        plot_smoothed_loss(axes[2], h, style['label'], k=100,␣
 ↪color=colors_3b[depth], ls=style['ls'])
axes[0].set_ylabel('training loss (mean over the epoch)'); axes[0].
 ↪set_title('Training loss vs epoch')
axes[1].set_ylabel('validation accuracy (%)'); axes[1].set_title('Validation␣
 ↪accuracy vs epoch')
axes[2].set_ylabel('training loss (moving average)'); axes[2].set_title('Depth␣
 ↪56: per-iteration training loss')
for ax in axes:
    ax.set_xlabel('epoch'); ax.legend(fontsize=8)
plt.tight_layout(); plt.show()

rows = []
for (depth, shortcut), h in results_3b.items():
    rows.append({'network': f'{"ResNet" if shortcut else "plain"}-{depth}',␣
 ↪'parameters': df_params.loc[depth, 'plain: parameters'],
```

```
                'train loss, epoch 1': h['train_loss'][0], 'final train loss':␣
  ↪h['train_loss'][-1],
                'final val acc (%)': 100 * h['val_acc'][-1], 'time (s)':␣
  ↪h['time']})
display(pd.DataFrame(rows).set_index('network').round(3))
```



network	parameters	train loss, epoch 1	final train loss	final val acc (%)	time (s)
plain-8	75290	1.768	0.477	82.04	213.663
ResNet-8	75290	1.773	0.470	83.12	219.556
plain-20	269722	1.834	0.339	85.46	625.439
ResNet-20	269722	1.744	0.256	88.04	635.265
plain-56	853018	2.275	0.989	65.20	1852.780
ResNet-56	853018	1.858	0.185	89.22	1872.420

Plain and residual networks of the same depth have identical parameter counts (75k, 270k, 853k) and nearly identical training times, so every difference above is caused by the shortcut alone.

- **Plain networks:** The final training loss is 0.477 (plain-8), 0.339 (plain-20) and 0.989 (plain-56). Plain-20 starts slower than plain-8 (loss 1.01 vs 0.95 at epoch 4) but overtakes it by epoch 6 and ends better (85.5% vs 82.0% validation accuracy). Plain-56 is far worse. Its epoch-1 loss of 2.275 is almost ln 10 2.30, so it barely learns at first, and after 20 epochs it is at a loss of 0.989 and 65.2% accuracy, still improving steeply. This is the **degradation problem**: it is the *training* loss that is worse, so it is not overfitting, and a 56-layer network could in principle represent anything a 20-layer one can. The difficulty is optimisation. Batch normalisation and Kaiming initialisation keep the signal at a reasonable scale, but SGD still cannot find a good solution.
- **Residual networks improve with depth.** The final training loss falls from 0.470 to

0.256 to 0.185, and the validation accuracy rises from 83.1% to 88.0% to 89.2%. Each block computes $x + F(x)$, so it only has to learn a correction, and "do nothing" is easy ($F \to 0$). The identity path also carries the gradient directly to the early layers. The returns diminish: going from 20 to 56 layers gains 1.2 pp for about 3× the parameters and 3× the training time (635 s → 1872 s).

- **The benefit of the shortcut grows with depth.** At depth 8 the two are almost the same. At depth 20 the benefit is modest (0.256 vs 0.339, +2.6 pp). At depth 56 it is large (0.185 vs 0.989, +24.0 pp). ResNet-56 passes a loss of 0.99 within the first 4 epochs, a level plain-56 reaches only at epoch 20. The shortcut changes how easily the network can be optimised, not its capacity, and it costs no extra parameters or time.

These are single 20-epoch runs. Plain-56 is still improving at the end, so a longer schedule would narrow the gap, but the point is that, with the same budget, the plain network cannot exploit its extra depth and the residual one can.

5 Task 4 — The best CNN with at most 500k parameters and 20 training epochs

Design choices

- **Residual network with BN** (the `CifarResNet` of 3(b)) with option-A identity shortcuts and global average pooling. Task 3(b) showed that the shortcuts make deep networks trainable, and global average pooling keeps the classifier tiny.
- **Depth vs width at a fixed budget.** In 3(b), residual networks improved with depth at fixed width (83.1% → 88.0% → 89.2% for ResNet-8/20/56). Three residual networks of 420–465k parameters are therefore compared on the validation set: a deep, narrow ResNet-32 (16-32-64), a ResNet-20 widened 1.25× (20-40-80) and a shallow, wide ResNet-8 (40-80-160).
- **Optimisation:** the recipe verified in 3(b): SGD with Nesterov momentum 0.9, learning rate 0.1, batch size 128, a 1-epoch linear warm-up and cosine decay to 0. Weight decay 5×10^{-4} is applied to conv and linear weights only, not to BN parameters or biases.
- **Regularisation:** flip + crop, which was by far the most effective regulariser in 3(a), plus a mild 8×8 cutout and label smoothing 0.1. Mixup and dropout are not used, because neither improved accuracy in 3(a) for this kind of network, and 3(a) showed that stacking regularisers can make a network under-fit within a short training budget.
- **Flip test-time augmentation (TTA):** the predictions for an image and its mirror image are averaged. This is used only if it improves the validation accuracy.

Procedure.
(1) Train the three candidates and choose the one with the best **validation** accuracy.
(2) Ablate four design choices on the chosen architecture, one at a time: the shortcut, the cutout, the weight decay and the label smoothing. Train the full recipe with two more seeds to measure the run-to-run noise.
(3) Decide on TTA using the validation set.
(4) Evaluate the final model **once** on the test set. Every run uses at most 20 epochs.

[17]:
```
T4_EPOCHS, T4_BS = 20, 128
# Three architectures with roughly the same parameter budget, from␣
  ↪deep-and-narrow to shallow-and-wide
T4_CANDIDATES = {
```

```
    'ResNet-32 (16-32-64)': dict(blocks=(5, 5, 5), widths=(16, 32, 64)),     # the 3(b) family, deepest that fits
    'ResNet-20 (20-40-80)': dict(blocks=(3, 3, 3), widths=(20, 40, 80)),     # ResNet-20, 1.25x wider
    'ResNet-8 (40-80-160)': dict(blocks=(1, 1, 1), widths=(40, 80, 160)),    # ResNet-8, 2.5x wider
}
# The 3(b) recipe (SGD, lr 0.1, batch 128, 1-epoch warm-up, cosine decay) plus Nesterov momentum,
# a mild 8x8 cutout, label smoothing, and weight decay 5e-4 on conv/linear weights only
BASE_RECIPE = dict(shortcut=True, flip_crop=True, cutout=8, weight_decay=5e-4, label_smoothing=0.1,
                   lr=0.1, warmup=0.05)

def sgd_no_decay_on_bn(model, lr, weight_decay):
    """Nesterov SGD; weight decay only on conv/linear weights (tensors with >1 dimension), not on BN or biases."""
    decay = [p for p in model.parameters() if p.ndim > 1]
    no_decay = [p for p in model.parameters() if p.ndim <= 1]
    return torch.optim.SGD([{'params': decay, 'weight_decay': weight_decay},
                            {'params': no_decay, 'weight_decay': 0.0}], lr=lr, momentum=0.9, nesterov=True)

def build_t4(recipe):
    return CifarResNet(recipe['blocks'], recipe['widths'], recipe['shortcut'])

def to_portable(state_dict):
    """Weights as plain Python lists, so a run trained on Colab loads on any other PyTorch/NumPy version."""
    return {k: (str(v.dtype).replace('torch.', ''), v.detach().cpu().tolist()) for k, v in state_dict.items()}

def from_portable(d):
    return {k: torch.tensor(v, dtype=getattr(torch, dt)) for k, (dt, v) in d.items()}

def run_t4(name, recipe, seed=SEED):
    def _run():
        seed_everything(seed)
        model = build_t4(recipe).to(device)
        n_params = count_params(model)
        assert n_params <= 500_000, f'{name}: {n_params} parameters exceed the 500k budget'
        opt = sgd_no_decay_on_bn(model, recipe['lr'], recipe['weight_decay'])
```

```
        hist = train(model, opt, T4_EPOCHS, X_train, Y_train, X_val, Y_val,␣
 ↪batch_size=T4_BS,
                     augment=make_augment(recipe['flip_crop'],␣
 ↪recipe['cutout']),
                     label_smoothing=recipe['label_smoothing'],
                     lr_schedule=cosine_schedule(warmup_frac=recipe['warmup']),␣
 ↪name=name)
        return dict(hist=hist, params=n_params, recipe=recipe,␣
 ↪state_dict=to_portable(model.state_dict()))
    return cached('task4_' + name + ('' if seed == SEED else f' seed{seed}'),␣
 ↪_run)

display(pd.DataFrame({name: {'blocks per stage': str(c['blocks']), 'channels':␣
 ↪str(c['widths']),
                              'parameters':␣
 ↪count_params(build_t4({**BASE_RECIPE, **c}))}
                      for name, c in T4_CANDIDATES.items()}).T)
```

	blocks per stage	channels	parameters
ResNet-32 (16-32-64)	(5, 5, 5)	(16, 32, 64)	464154
ResNet-20 (20-40-80)	(3, 3, 3)	(20, 40, 80)	420670
ResNet-8 (40-80-160)	(1, 1, 1)	(40, 80, 160)	464690

[18]:
```
# ---- step 1: choose the architecture on the VALIDATION set ----
cand_results = {name: run_t4(name, {**BASE_RECIPE, **c}) for name, c in␣
 ↪T4_CANDIDATES.items()}

df_cand = pd.DataFrame({name: {'parameters': r['params'], 'final val acc (%)':␣
 ↪100 * r['hist']['val_acc'][-1],
                                'final train loss': r['hist']['train_loss'][-1],␣
 ↪'time (s)': r['hist']['time']}
                        for name, r in cand_results.items()}).T
display(df_cand.round(3))
best_name = max(cand_results, key=lambda k:␣
 ↪cand_results[k]['hist']['val_acc'][-1])
best_recipe = cand_results[best_name]['recipe']
print('chosen architecture:', best_name)

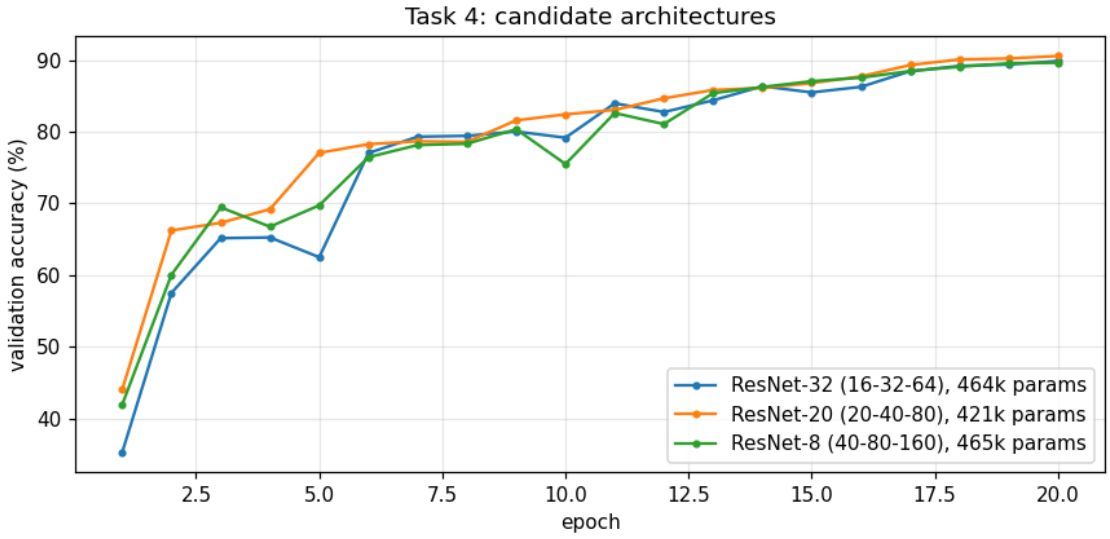
fig, ax = plt.subplots(figsize=(8, 4))
for name, r in cand_results.items():
    ax.plot(np.arange(1, len(r['hist']['val_acc']) + 1), 100 * np.
 ↪array(r['hist']['val_acc']), 'o-', ms=3,
            label=f'{name}, {r["params"] / 1e3:.0f}k params')
ax.set_xlabel('epoch'); ax.set_ylabel('validation accuracy (%)'); ax.
 ↪set_title('Task 4: candidate architectures'); ax.legend()
plt.tight_layout(); plt.show()
```

```
[cache] loaded results\task4_ResNet-32_16-32-64.pkl  (delete it to re-run)
[cache] loaded results\task4_ResNet-20_20-40-80.pkl  (delete it to re-run)
[cache] loaded results\task4_ResNet-8_40-80-160.pkl  (delete it to re-run)

                      parameters  final val acc (%)  final train loss  \
ResNet-32 (16-32-64)    464154.0              89.88             0.701
ResNet-20 (20-40-80)    420670.0              90.58             0.699
ResNet-8 (40-80-160)    464690.0              89.64             0.774

                      time (s)
ResNet-32 (16-32-64)   180.433
ResNet-20 (20-40-80)   142.690
ResNet-8 (40-80-160)   108.924

chosen architecture: ResNet-20 (20-40-80)
```



[19]:
```python
# ---- step 2: ablations on the chosen architecture (one change at a time) ----
ABLATIONS = {
    'no shortcut (plain network)':  dict(shortcut=False),
    'no cutout (flip + crop only)': dict(cutout=0),
    'no weight decay':              dict(weight_decay=0.0),
    'no label smoothing':           dict(label_smoothing=0.0),
}
abl_results = {'full model': cand_results[best_name]}
for abl_name, change in ABLATIONS.items():
    abl_results[abl_name] = run_t4(f'{best_name} | {abl_name}', {**best_recipe,
 ↪**change})

# run-to-run noise of the full recipe: same data split, different
 ↪initialisation and batch order
```

```
seed_accs = [100 * cand_results[best_name]['hist']['val_acc'][-1]] + \
            [100 * run_t4(best_name, best_recipe,␣
 ↪seed=s)['hist']['val_acc'][-1] for s in (1, 2)]
seed_mean, seed_std = np.mean(seed_accs), np.std(seed_accs, ddof=1)
print(f'full recipe over 3 seeds: {seed_mean:.2f} ± {seed_std:.2f} % '
      f'(individual runs: {", ".join(f"{a:.2f}" for a in seed_accs)})')

full_acc = 100 * abl_results['full model']['hist']['val_acc'][-1]
df_abl = pd.DataFrame({name: {'parameters': r['params'], 'val acc (%)': 100 *␣
 ↪r['hist']['val_acc'][-1],
                               'Δ vs full (pp)': 100 * r['hist']['val_acc'][-1]␣
 ↪- full_acc,
                               'Δ vs 3-seed mean (pp)': 100 *␣
 ↪r['hist']['val_acc'][-1] - seed_mean,
                               'final train loss': r['hist']['train_loss'][-1]}
                       for name, r in abl_results.items()}).T
display(df_abl.round(3))

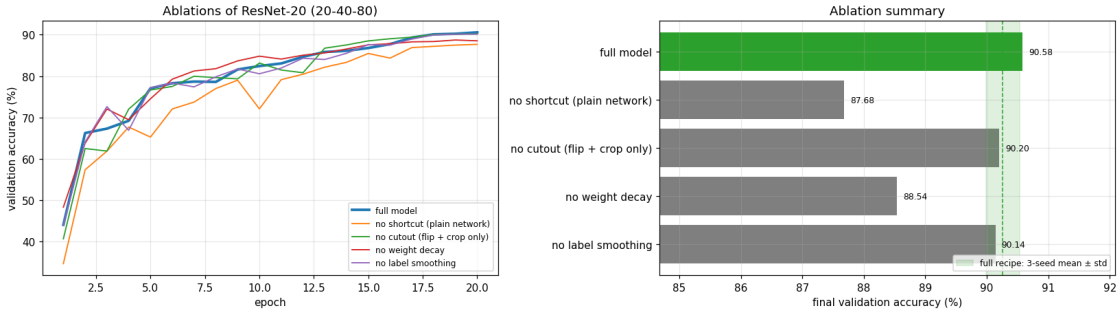
fig, axes = plt.subplots(1, 2, figsize=(15, 4.3))
for name, r in abl_results.items():
    axes[0].plot(np.arange(1, len(r['hist']['val_acc']) + 1), 100 * np.
 ↪array(r['hist']['val_acc']),
                 lw=2.5 if name == 'full model' else 1.2, label=name)
axes[0].set_xlabel('epoch'); axes[0].set_ylabel('validation accuracy (%)');␣
 ↪axes[0].legend(fontsize=8)
axes[0].set_title(f'Ablations of {best_name}')
axes[1].barh(df_abl.index[::-1], df_abl['val acc (%)'][::-1], color=['tab:
 ↪gray'] * (len(df_abl) - 1) + ['tab:green'])
for i, v in enumerate(df_abl['val acc (%)'][::-1]):
    axes[1].text(v + 0.1, i, f'{v:.2f}', va='center', fontsize=8)
axes[1].set_xlim(df_abl['val acc (%)'].min() - 3, df_abl['val acc (%)'].max() +␣
 ↪1.5)
axes[1].set_xlabel('final validation accuracy (%)'); axes[1].
 ↪set_title('Ablation summary')
axes[1].axvspan(seed_mean - seed_std, seed_mean + seed_std, color='tab:green',␣
 ↪alpha=0.15,
                label='full recipe: 3-seed mean ± std')
axes[1].axvline(seed_mean, color='tab:green', ls='--', lw=1)
axes[1].legend(fontsize=8, loc='lower right')
plt.tight_layout(); plt.show()
```

```
[cache] loaded results\task4_ResNet-20_20-40-80_no_shortcut_plain_network.pkl
(delete it to re-run)
[cache] loaded results\task4_ResNet-20_20-40-80_no_cutout_flip_+_crop_only.pkl
(delete it to re-run)
[cache] loaded results\task4_ResNet-20_20-40-80_no_weight_decay.pkl  (delete it
to re-run)
```

```
[cache] loaded results\task4_ResNet-20_20-40-80_no_label_smoothing.pkl  (delete
it to re-run)
[cache] loaded results\task4_ResNet-20_20-40-80_seed1.pkl  (delete it to re-run)
[cache] loaded results\task4_ResNet-20_20-40-80_seed2.pkl  (delete it to re-run)
full recipe over 3 seeds: 90.26 ± 0.28 % (individual runs: 90.58, 90.10, 90.10)
```

	parameters	val acc (%)	Δ vs full (pp)	\
full model	420670.0	90.58	0.00	
no shortcut (plain network)	420670.0	87.68	-2.90	
no cutout (flip + crop only)	420670.0	90.20	-0.38	
no weight decay	420670.0	88.54	-2.04	
no label smoothing	420670.0	90.14	-0.44	

	Δ vs 3-seed mean (pp)	final train loss
full model	0.32	0.699
no shortcut (plain network)	-2.58	0.745
no cutout (flip + crop only)	-0.06	0.645
no weight decay	-1.72	0.752
no label smoothing	-0.12	0.223



The chosen architecture is ResNet-20 (20-40-80) with 420,670 parameters, reaching 90.58% validation accuracy. The three candidates were within 1 pp (89.64–90.58%), so depth and width matter little at this budget. Over three seeds the full recipe gives **90.26 ± 0.28 %**, so the ablations are compared with this mean, and only effects clearly larger than 0.28 pp count as real.

- **Shortcut** (−2.58 pp). The plain network lags throughout and is less stable, matching the 2.6 pp gap at depth 20 in Task 3(b). The shortcut makes optimisation easier.
- **Weight decay** (−1.72 pp). Without it the network is ahead early (84.8% at epoch 10) but stalls at 88.5%. In a BN network, growing weights lower the effective learning rate, so training slows down. Weight decay keeps the effective step size larger, which also regularises.
- **Cutout** (−0.06 pp) and **label smoothing** (−0.12 pp). Both are within the noise: flip + crop already gives most of the augmentation benefit, and the network is not overfitting (validation loss still falling at epoch 20). Compared with the single full-model run they look like −0.4 pp, which shows that one run cannot judge small effects.

The components that change how the network is *optimised* (shortcut, weight decay) matter far more than the extra regularisers.

[20]:
```
# ---- step 3: test-time augmentation (flip), decided on the VALIDATION set ----
@torch.no_grad()
def predict(model, X, tta=False, batch_size=1000):
    """Class predictions; with tta=True the softmax outputs for the image and␣
 ↪its mirror image are averaged."""
    model.eval()
    preds = []
    for k in range(0, len(X), batch_size):
        xb = X[k:k + batch_size]
        with autocast_ctx():
            out = model(xb).float()
            if tta:
                out = out.softmax(1) + model(xb.flip(3)).float().softmax(1)
        preds.append(out.argmax(1))
    return torch.cat(preds)

final = cand_results[best_name]
final_model = build_t4(final['recipe']).to(device)
final_model.load_state_dict(from_portable(final['state_dict']))
val_acc_plain = (predict(final_model, X_val) == Y_val).float().mean().item()
val_acc_tta = (predict(final_model, X_val, tta=True) == Y_val).float().mean().
 ↪item()
USE_TTA = val_acc_tta > val_acc_plain
print(f'validation accuracy without TTA: {100 * val_acc_plain:.2f}%   with flip␣
 ↪TTA: {100 * val_acc_tta:.2f}%')
print('-> flip TTA', 'IS' if USE_TTA else 'is NOT', 'used for the final test␣
 ↪evaluation')
```

```
validation accuracy without TTA: 90.58%   with flip TTA: 91.14%
-> flip TTA IS used for the final test evaluation
```

Flip test-time augmentation (averaging the softmax outputs of each image and its mirror image) raised the validation accuracy from 90.58% to 91.14% (+0.56 pp), so it is used for the test evaluation. This decision was made on the validation set only.

5.1 Final evaluation on the test set

The final model is the chosen architecture, ResNet-20 (20-40-80), trained with the full recipe above (the run with the seed used throughout the notebook): 20 epochs on the 45k training images, with the architecture selected using the validation set only. Flip test-time augmentation (averaging the softmax outputs of each image and its mirror image) raised the validation accuracy from 90.58% to 91.14%, so it is used for the test evaluation; this decision was also made on the validation set only. The test set is loaded, normalised with the *training* statistics and used **exactly once**, in the cell below. The overall accuracy and the confusion matrix both come from this single pass.

[21]:
```
# ======== THE ONLY USE OF THE CIFAR-10 TEST SET IN THIS NOTEBOOK ========
cifar_test = torchvision.datasets.CIFAR10(DATA_ROOT, train=False, download=True)
```

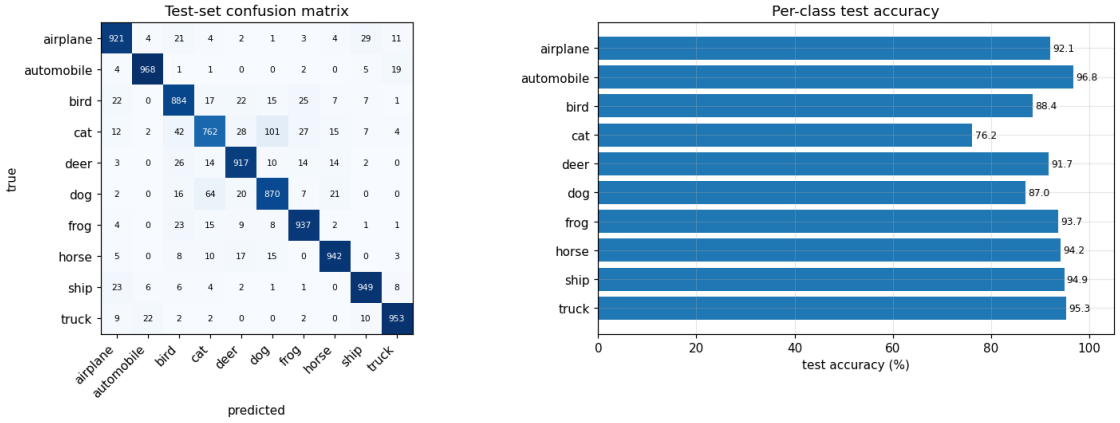
```
X_test = normalize(torch.tensor(cifar_test.data).permute(0, 3, 1, 2).float().
 ↪div(255)).to(device)
Y_test = torch.tensor(cifar_test.targets, dtype=torch.long).to(device)

test_pred = predict(final_model, X_test, tta=USE_TTA)      # final_model and␣
 ↪USE_TTA come from step 3
test_acc = (test_pred == Y_test).float().mean().item()

print(f'Final model    : {best_name}, {final["params"]:,} parameters,␣
 ↪{T4_EPOCHS} training epochs'
      + (', flip TTA' if USE_TTA else ''))
print(f'Validation acc : {100 * (val_acc_tta if USE_TTA else val_acc_plain):.
 ↪2f}%')
print(f'TEST accuracy  : {100 * test_acc:.2f}%')

cm = torch.bincount(Y_test * 10 + test_pred, minlength=100).view(10, 10).cpu().
 ↪numpy()
per_class = cm.diagonal() / cm.sum(1)
fig, axes = plt.subplots(1, 2, figsize=(14, 5), gridspec_kw={'width_ratios': [1.
 ↪25, 1]})
im = axes[0].imshow(cm, cmap='Blues')
axes[0].set_xticks(range(10)); axes[0].set_xticklabels(CLASSES, rotation=45,␣
 ↪ha='right')
axes[0].set_yticks(range(10)); axes[0].set_yticklabels(CLASSES); axes[0].
 ↪grid(False)
for i in range(10):
    for j in range(10):
        axes[0].text(j, i, cm[i, j], ha='center', va='center', fontsize=7,␣
 ↪color='white' if cm[i, j] > cm.max() / 2 else 'black')
axes[0].set_xlabel('predicted'); axes[0].set_ylabel('true'); axes[0].
 ↪set_title('Test-set confusion matrix')
axes[1].barh(CLASSES[::-1], 100 * per_class[::-1])
for i, v in enumerate(100 * per_class[::-1]):
    axes[1].text(v + 0.5, i, f'{v:.1f}', va='center', fontsize=8)
axes[1].set_xlabel('test accuracy (%)'); axes[1].set_title('Per-class test␣
 ↪accuracy'); axes[1].set_xlim(0, 105)
plt.tight_layout(); plt.show()
```

```
Final model    : ResNet-20 (20-40-80), 420,670 parameters, 20 training epochs,
flip TTA
Validation acc : 91.14%
TEST accuracy  : 91.03%
```



The final model, ResNet-20 (20-40-80) with flip TTA, reaches **91.03 %** test accuracy, 0.11 pp below its validation accuracy of 91.14 %. This is well within the sampling noise of the two sets (standard error about 0.3 pp on the 10,000 test images and 0.4 pp on the 5,000 validation images), so choosing the architecture and TTA on the validation split did not overfit it noticeably.

The errors are concentrated in a few visually similar pairs: * **Cat <-> dog** is the largest confusion by far: 101 cats are predicted as dogs and 64 dogs as cats (165 errors, 18% of all 897). Cat is the weakest class (76.2 %) and dog the third weakest (87.0 %). Cats are also confused with bird (42), deer (28) and frog (27). At 32×32 pixels, the details that separate these animals (ear shape, snout) are only a few pixels wide, and they share fur texture, colours and poses. * **Vehicles** are recognised best (automobile 96.8 %, truck 95.3 %, ship 94.9 %), except airplane (92.1 %). The remaining vehicle errors are between similar pairs: airplane <-> ship (29 and 23, likely because of similar blue backgrounds) and automobile <-> truck (19 and 22). * The animal classes average 88.5 % against 94.8 % for the vehicles, although frog (93.7 %) and horse (94.2 %) are recognised nearly as well as the vehicles.

Most remaining errors are therefore fine-grained distinctions between similar animals.

6 Conclusions

1. **Backpropagation (Task 1).** The hand-written backward passes of `Linear`, `ReLU`, `Sigmoid` and softmax cross-entropy match autograd to machine precision (relative error 4×10^{-17} to 2×10^{-16}) and match centred finite differences to 5×10^{-10}–5×10^{-7}. An MLP trained with them reaches 51.2% validation accuracy on flattened CIFAR-10 (best 52.6% at epoch 10). It overfits heavily (94% training accuracy, validation loss rising from 1.40 at epoch 4 to 2.10), because flattening discards spatial structure and the 1.7M parameters are unregularised. In a 10-layer sigmoid network the gradient at layer 1 is 1.5×10^5 (Xavier) and 1.4×10^4 (Kaiming) times smaller than at layer 10, since $\sigma' \leq 1/4$. ReLU with Kaiming initialisation keeps the gradient constant across layers. ReLU with Xavier also looks flat across layers, but its overall scale is about 15–45 times lower, because the forward and backward shrinkage cancel in the weight gradients.

2. **Optimizers (Task 2).** My SGD-with-momentum and Adam implementations matched results with `torch.optim`. On the ill-conditioned quadratic $\frac{1}{2}(x^2 + 50y^2)$, plain SGD zig-zags

across the valley and crawls along the flat direction (final $f = 4 \times 10^{-2}$). Momentum and Adam end about 28 times lower (1.5×10^{-3} and 1.4×10^{-3}): momentum by building up speed along the flat direction (and overshooting), Adam by rescaling each coordinate. On the CNN, every optimizer works well only within about one decade of learning rates. The best settings are SGD at 0.1 (70.0%), momentum at 0.01 (73.8%) and Adam at 10^{-3} (74.2%). Momentum collapses to chance at 0.1, and Adam, whose default worked without tuning, drops to 54.2% at 10^{-2}.

3. **Regularisation (Task 3a).** On 5k images the baseline memorises the training set (100% training, 65.5% validation, a 34.5 pp gap). Flip + crop helped most (+10.4 pp, 75.9%), because it encodes invariances that really hold for images. Dropout (+3.7 pp) and cutout (+2.8 pp) gave moderate gains, weight decay was within noise (+0.7 pp), and mixup lowered the validation loss (1.12 vs 1.62) but not the error rate (−1.9 pp). Combining everything gave the smallest gap (10.2 pp) but not the best accuracy (67.1%), because the network under-fits (77% training accuracy). The gap is a diagnostic, not the goal.

4. **Residual connections (Task 3b).** Plain and residual networks of the same depth have identical parameter counts, so every difference comes from the shortcut. Plain networks degrade only at depth 56: the final training loss is 0.99 for plain-56, against 0.34 for plain-20, so this is an optimisation failure, not overfitting. With shortcuts, deeper is better: the validation accuracy rises from 83.1% to 88.0% to 89.2% for depths 8, 20 and 56. The benefit of the shortcut grows with depth (+1.1 pp at depth 8, +2.6 pp at depth 20 and +24.0 pp at depth 56).

5. **Best CNN (Task 4).** A 420,670-parameter ResNet-20 (20-40-80), trained for 20 epochs, reaches **91.03%** test accuracy (91.14% validation) with flip test-time augmentation, which added 0.56 pp on validation. At this parameter budget, depth and width mattered little: the three candidates were within 1 pp. Over three seeds the full recipe gives 90.26 ± 0.28% on validation. The ablations showed that the shortcuts (−2.6 pp) and weight decay (−1.7 pp) mattered most, while cutout and label smoothing had no measurable effect (−0.06 and −0.12 pp, within the noise). Flip + crop was kept in every run, because Task 3(a) showed it was the most effective regulariser. Most remaining errors are cat <-> dog confusions.

Overall. Across all tasks, the components that make *optimisation* work (ReLU with a variance-preserving initialisation, shortcut connections, weight decay, a sensible learning rate) mattered more than additional regularisers, once the data augmentation was in place.

6.1 References

- K. He, X. Zhang, S. Ren, J. Sun. *Deep Residual Learning for Image Recognition.* CVPR 2016.
- K. He, X. Zhang, S. Ren, J. Sun. *Delving Deep into Rectifiers.* ICCV 2015 (He / Kaiming initialisation).
- X. Glorot, Y. Bengio. *Understanding the difficulty of training deep feedforward neural networks.* AISTATS 2010.
- D. P. Kingma, J. Ba. *Adam: A Method for Stochastic Optimization.* ICLR 2015.
- T. DeVries, G. W. Taylor. *Improved Regularization of Convolutional Neural Networks with Cutout.* arXiv:1708.04552, 2017.
- H. Zhang, M. Cisse, Y. N. Dauphin, D. Lopez-Paz. *mixup: Beyond Empirical Risk Minimization.* ICLR 2018.